Attention and transformers

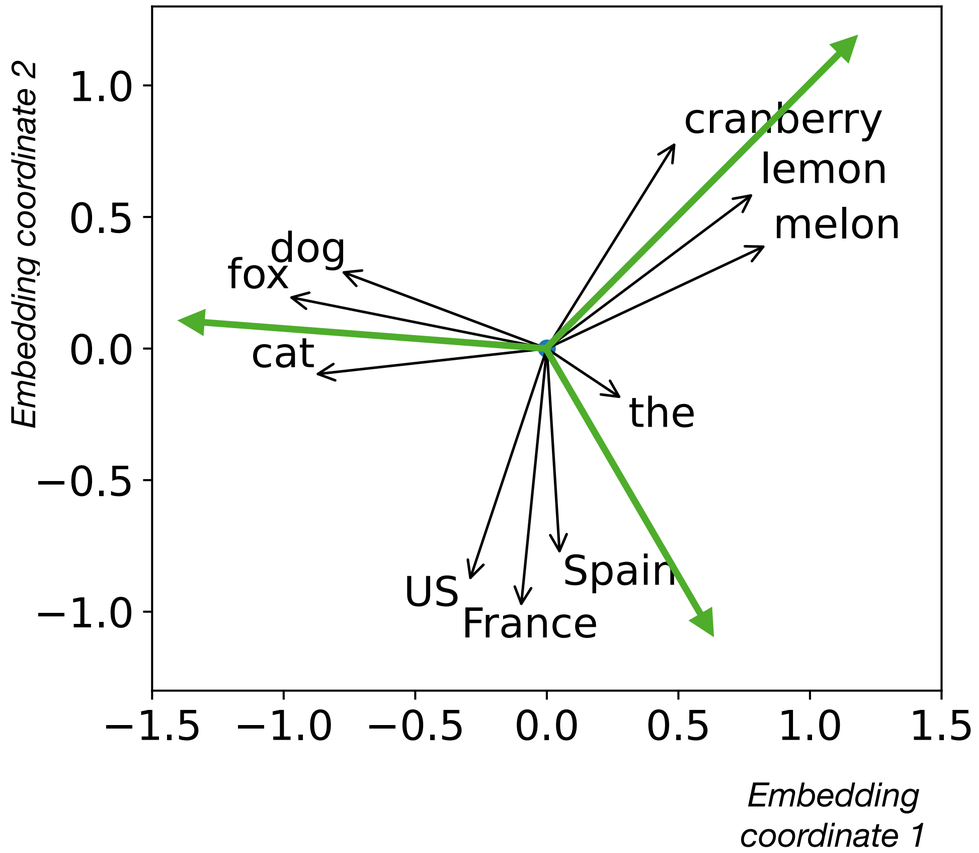
10-701 Introduction to Machine Learning

Geoff Gordon

Beyond word2vec

In the figure,, each of the three green queries retrieves primarily tokens for one group of concepts — e.g., {fox, dog, cat} on the left

- Embedding and attention are useful primitives, used in many more models beyond word2vec
- Interpret some hidden activation vectors as a database (like word2vec vocabulary)
  - $v_1, v_2, \ldots, v_K \in \mathbb{R}^d$



- Interpret other hidden activation vectors as a sequence of queries (like word2vec sliding window on input)
  - $q_1, q_2, \ldots, q_T \in \mathbb{R}^d$
- Can use networks with attention for all the sequence tasks mentioned above (many:many, many:1, …)

Attention (initial version)

- Look up queries in DB: for each query q_i
  - find attention weights over DB entries:

$$p_{ij} = \frac{\exp(\frac{1}{\sqrt{d}} q_i \cdot v_j)}{\sum_{j'} \exp(\frac{1}{\sqrt{d}} q_i \cdot v_{j'})}$$

  - return weighted average of DB entries:

$$\text{output}_i = \sum_j p_{ij} v_j$$

- Note scaling factor $1/\sqrt{d}$
  - doesn't change expressive power: can compensate by scaling q_i or v_j
  - but makes initialization less sensitive to query dimension

Attention: queries, keys, and values

$q_i, k_j \in \mathbb{R}^d \quad v_j \in \mathbb{R}^k$

- Generalization: DB contains *key-value pairs* (k_j, v_j)
  - compare query to k_j but return v_j in case of match
- Compare query q_i to key k_j:

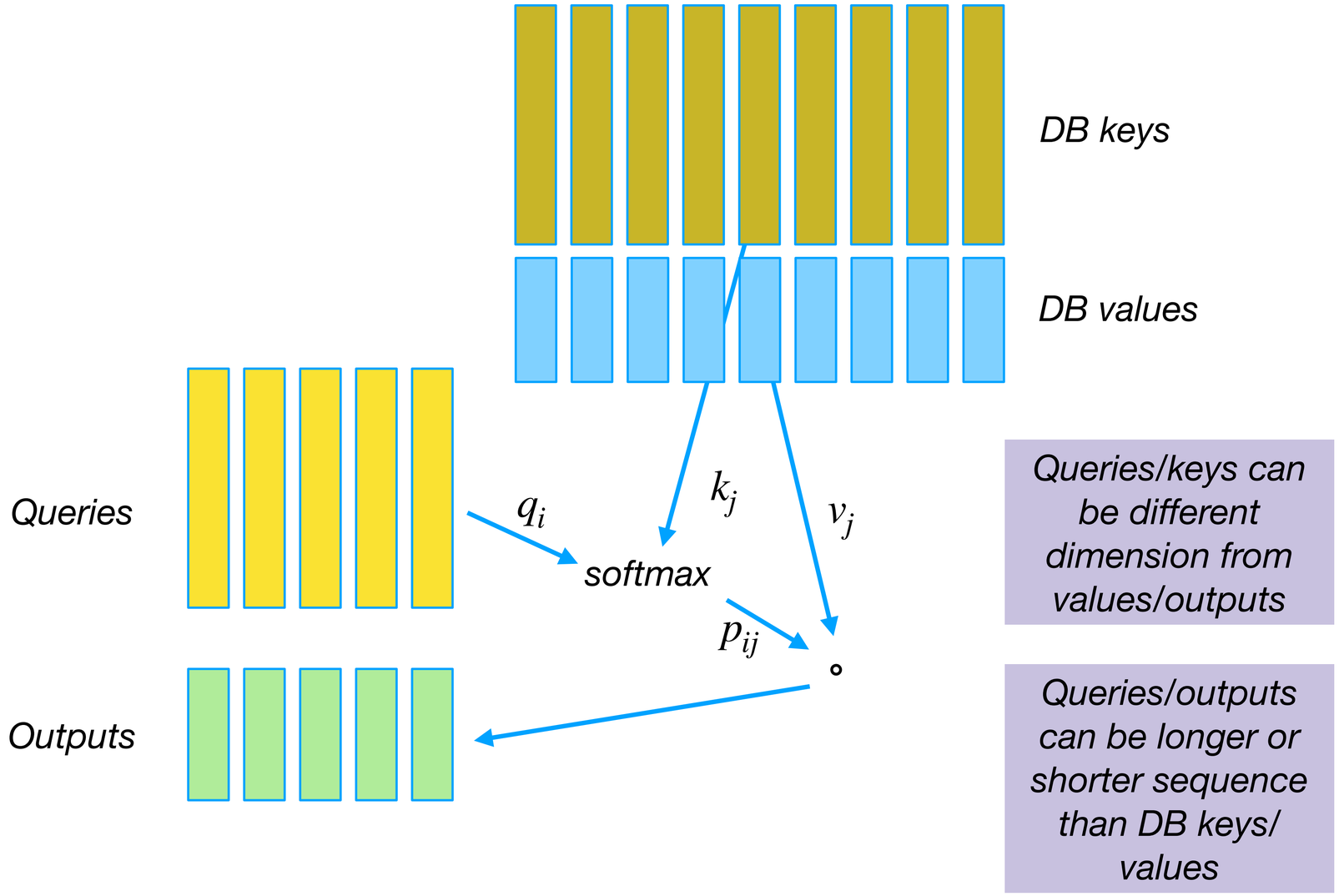
$$\text{attention weights } p_{ij} = \frac{\exp(\frac{1}{\sqrt{d}} q_i \cdot k_j)}{\sum_{j'} \exp(\frac{1}{\sqrt{d}} q_i \cdot k_{j'})}$$

- Return weighted average of values v_j:

$$\text{output}_i = \sum_j p_{ij} v_j$$

- Decouple what we look up from what we return
  - v_j can even be a different dimension

Attention



- Called an *attention head:* maps sequence of queries to sequence of outputs, using DB (key-value pairs) as reference

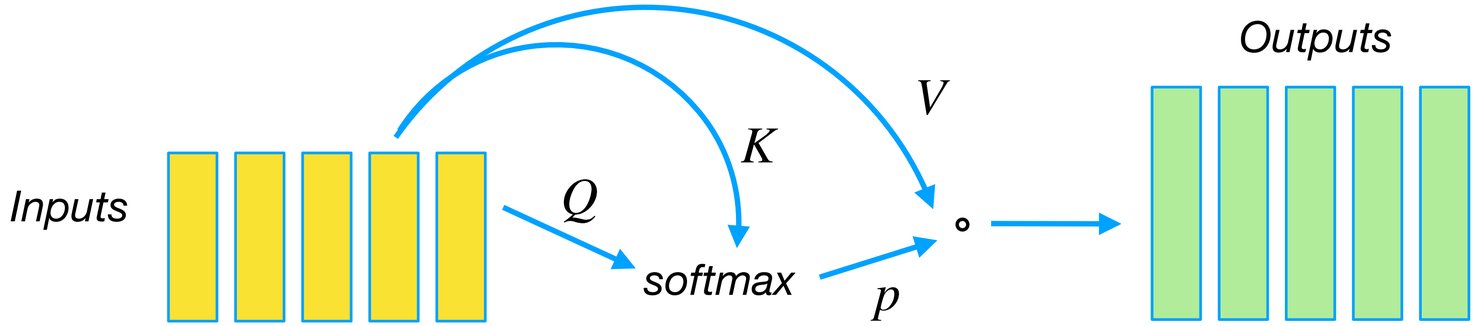
Self-attention

- Single sequence $x_i \in \mathbb{R}^d$ (e.g., input tokens, or output of previous layer) acts as *both* DB and queries
- Transform with learnable parameter matrices Q, K, V
  - queries $q_i = Qx_i$, keys $k_i = Kx_i$, values $v_i = Vx_i$
- Once we get DB and queries from x_i, proceed as before

$$p_{ij} = \frac{\exp(Qx_i \cdot Kx_j)}{\sum_{j'} \exp(Qx_i \cdot Kx_{j'})} \qquad \text{output}_i = \sum_j p_{ij} V x_j$$

- Queries q_i and keys k_i have to be same dimension, but can differ from dimension of inputs x_i and outputs v_i
  - Q, K map x_i to a common dimension for dot product; V maps x_i to desired output dimension

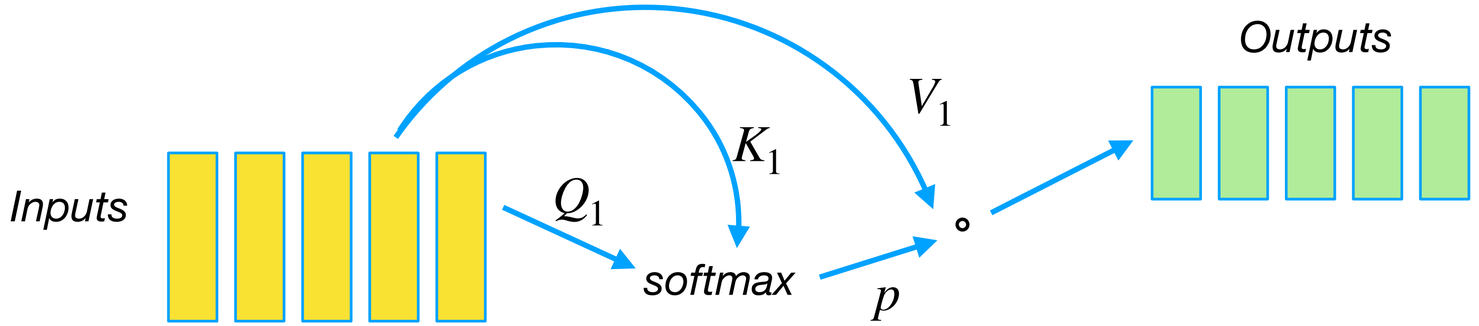
Self-attention



John went to the grocery store , he bought an apple

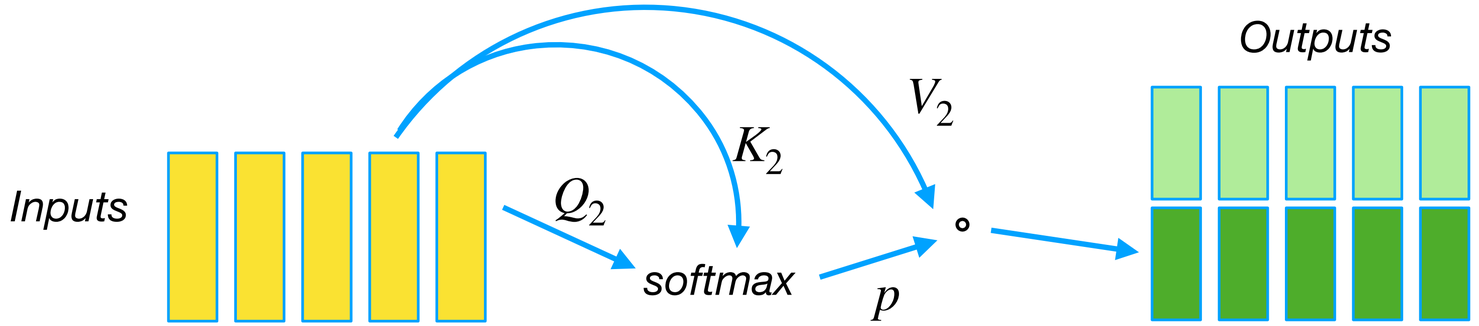
- Database and query sequence come from same source:
  - for each token in a document, find other relevant tokens
  - e.g., referent of a pronoun [he → John]
  - e.g., similar topic [apple → grocery, store]
- Extremely parallelizable! :) But $O(T^2)$:(

Multi-head attention



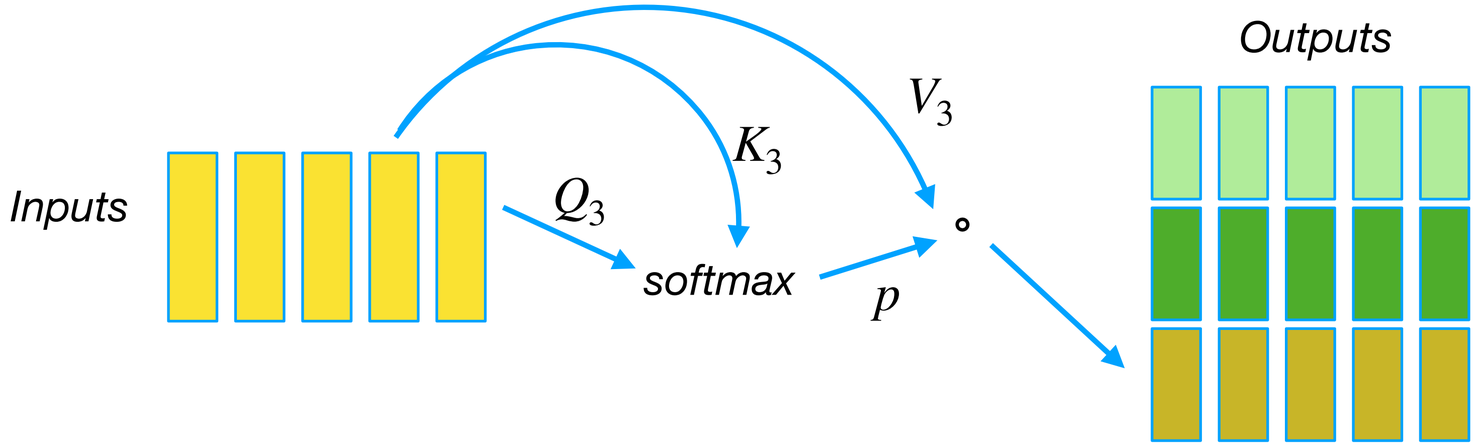
- Do attention several times in parallel, concatenate results
  - often make each head's output value shorter so total dimension remains the same

Multi-head attention



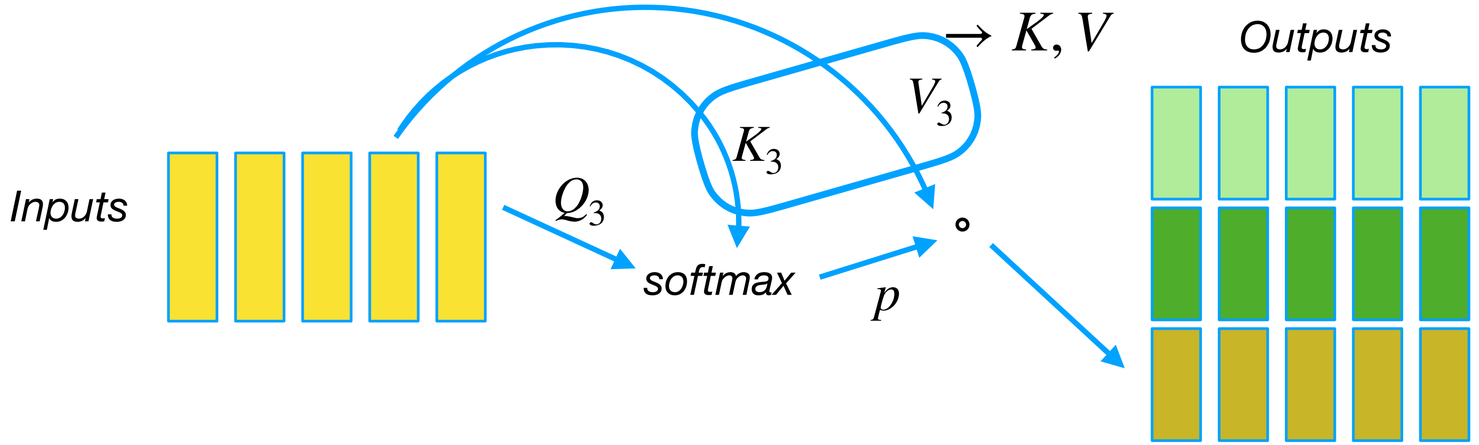
- Do attention several times in parallel, concatenate results
  - often make each head's output value shorter so total dimension remains the same

Multi-head attention



- Do attention several times in parallel, concatenate results
  - often make each head's output value shorter so total dimension remains the same

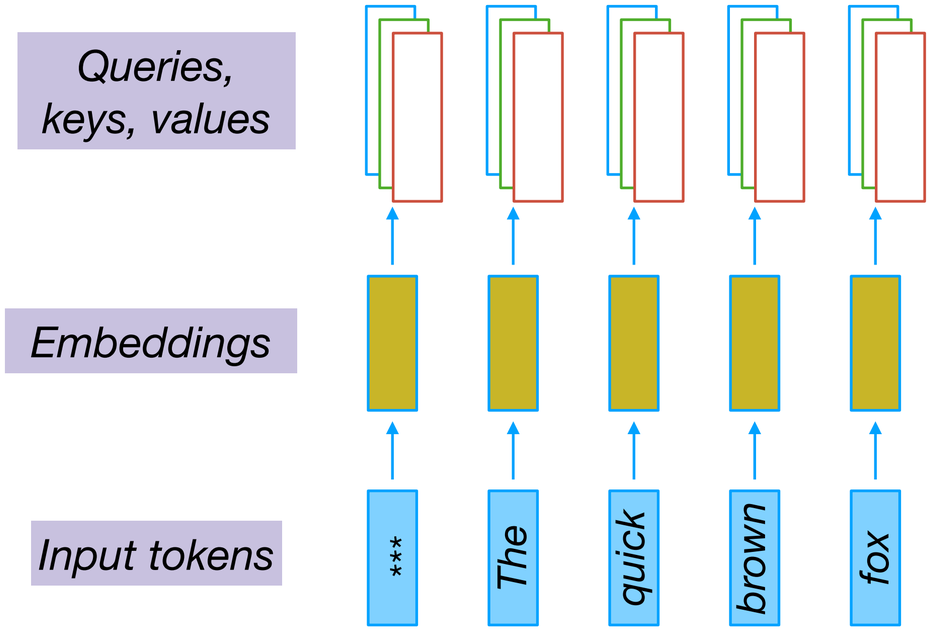
GQA



- Can share parameters: e.g., keep K, V constant in blocks while Q varies (***grouped query attention***, GQA)
  - saves memory, computation, and model size

Positional encoding

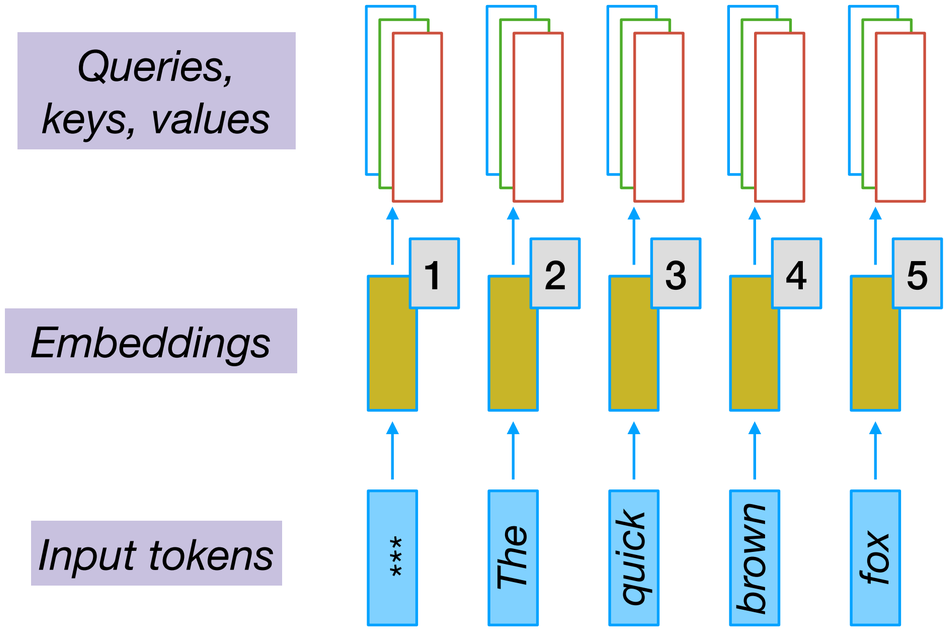
- Attention is permutation-invariant for DB, equivariant for queries
  - order of DB doesn't affect prediction
  - reordering queries reorders predictions the same way



- Often want only sort-of-invariance: *generalize* from position to position, but don't *force* identical behavior
- Solution: encode token position somehow in embedding or attention layer
  - network can pay attention or ignore, whatever works best

Positional encoding

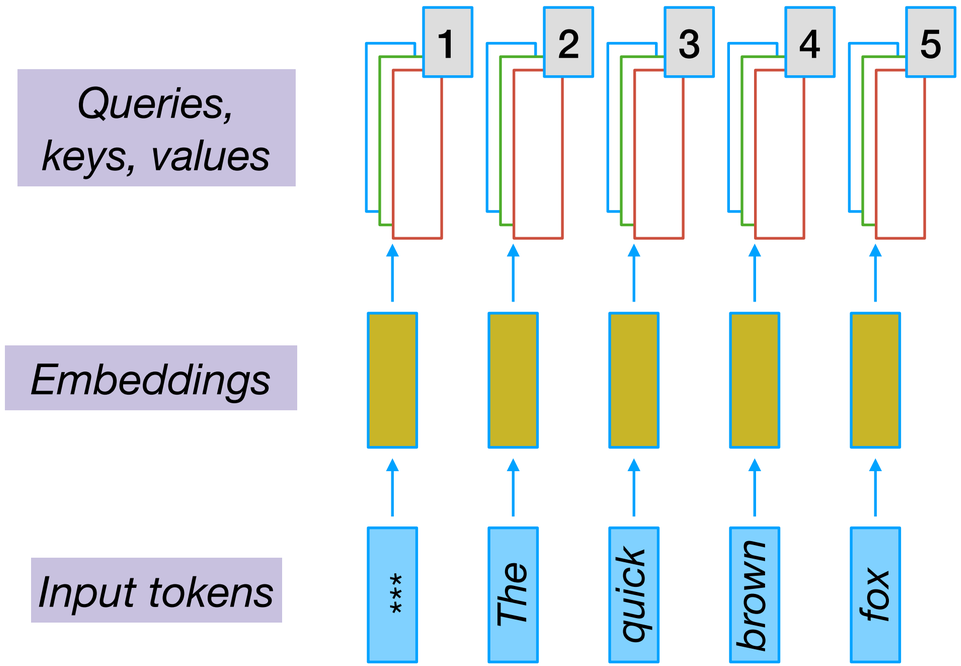
- Attention is permutation-invariant for DB, equivariant for queries
  - order of DB doesn't affect prediction
  - reordering queries reorders predictions the same way



- Often want only sort-of-invariance: *generalize* from position to position, but don't *force* identical behavior
- Solution: encode token position somehow in embedding or attention layer
  - network can pay attention or ignore, whatever works best

Positional encoding

- Attention is permutation-invariant for DB, equivariant for queries
  - order of DB doesn't affect prediction
  - reordering queries reorders predictions the same way



- Often want only sort-of-invariance: *generalize* from position to position, but don't *force* identical behavior
- Solution: encode token position somehow in embedding or attention layer
  - network can pay attention or ignore, whatever works best

What makes a good positional encoding?

- Consider a query q_i at position i and key k_j at position j
  - imagine varying j: keep the same query and key, but the key shows up at different positions
  - a positional encoding at attention layer alters $q_i \to q_i^{\text{enc}}$ and $k_j \to k_j^{\text{enc}}$
  - which lets the dot product $q_i^{\text{enc}} \cdot k_j^{\text{enc}}$ vary as we shift j
  - therefore attention weight p_{ij} also varies
- An ideal positional encoding would let us represent any translation-invariant score function $q_i^{\text{enc}} \cdot k_j^{\text{enc}} = f(j - i)$
  - attend iff k shows up at previous token, or attend iff k shows up at position +10 to +15, or …

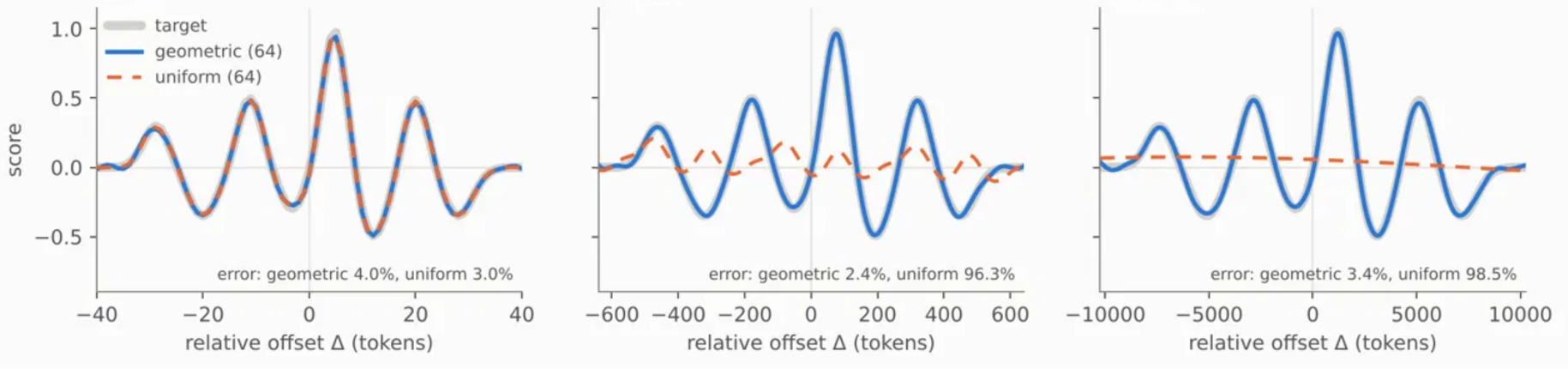
Representing functions: Fourier basis

- One good way to represent functions is a Fourier basis:

$$f(\Delta) \approx \sum_{b=1}^{d} \alpha_b \sin(\theta_b \Delta) + \beta_b \cos(\theta_b \Delta) \qquad \Delta = j - i$$

  - with d different (fixed) frequencies θ_b, weights α_b, β_b
- Which frequencies?
  - different choices lead to different representational bias
  - Fourier transform uses uniform sampling of frequencies: $\frac{2\pi}{N}, \frac{2(2\pi)}{N}, \frac{3(2\pi)}{N}, \ldots$ for a function with inputs $0 \ldots N-1$
  - could also do geometric (uniform on a log scale): $B^{\frac{1}{N}}, B^{\frac{2}{N}}, B^{\frac{3}{N}}, \ldots$ for some base $B < 1$
  - many other possible choices

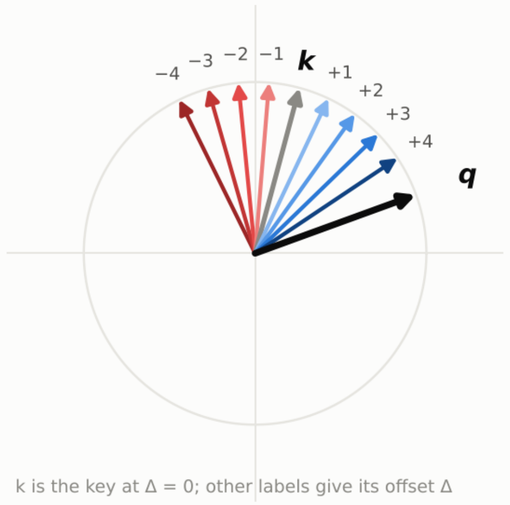
Effect of frequency sampling

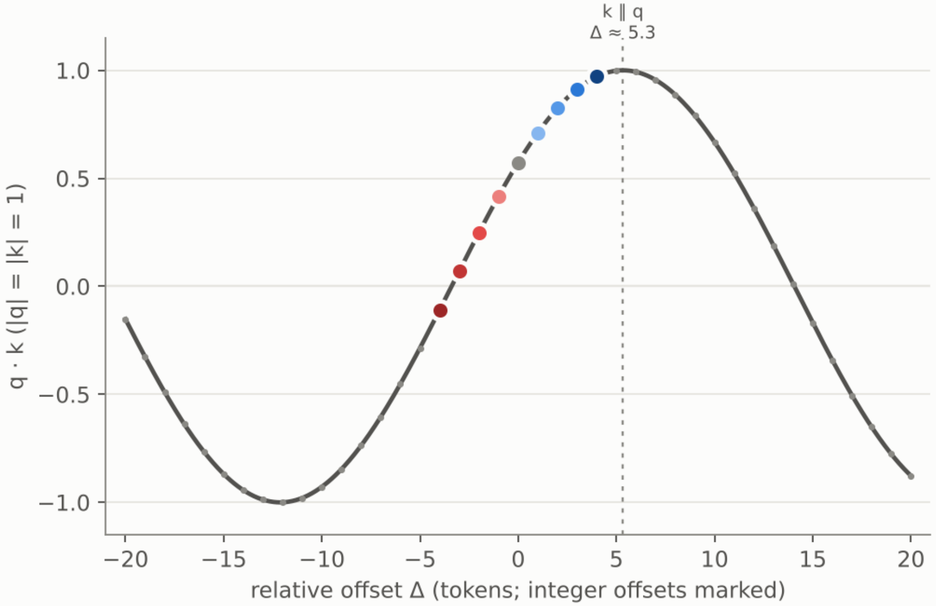


- We'll use geometric: gives *scale-invariant* accuracy
  - error for representing a function $f(\Delta)$ is about the same as for $f(\alpha\Delta)$ for any scale $\alpha > 0$ in a wide range
- Plot: given a target $f(\Delta)$ that is nonzero in –40..40,
  - show best fit for f in Fourier basis with geometric vs. uniform sampling
  - compare after scaling f to cover –600..600 or –10k..10k

RoPE: one frequency

how do we get sin or cos out of dot product of embeddings?





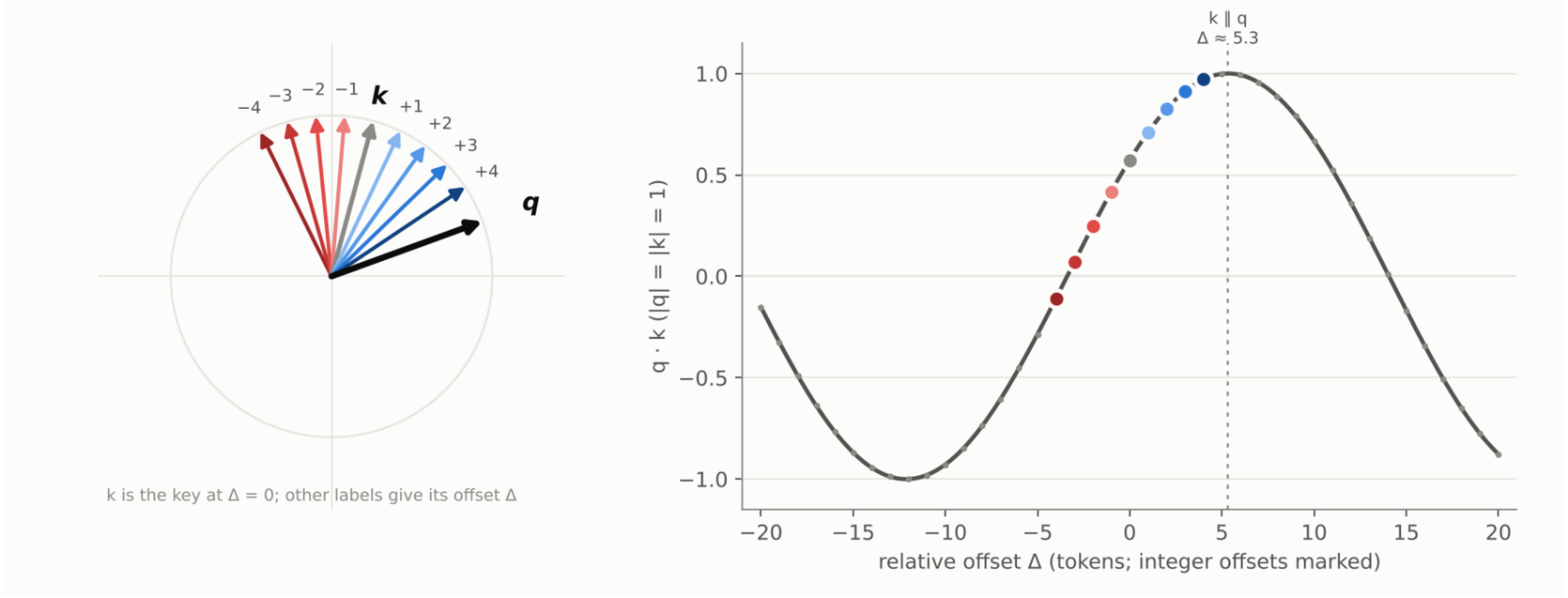
- Suppose $q, k \in \mathbb{R}^2$, with q at token position i and k at j
- Pick a fixed angle θ
- Transform by rotating in increments of θ
  - $q^{\text{enc}} = R_{i\theta} q$ and $k^{\text{enc}} = R_{j\theta} k$
  - plot: fix i, sweep j

$$R_\theta = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$

rotates CCW by θ

RoPE: one frequency

how do we get sin or cos out of dot product of embeddings?



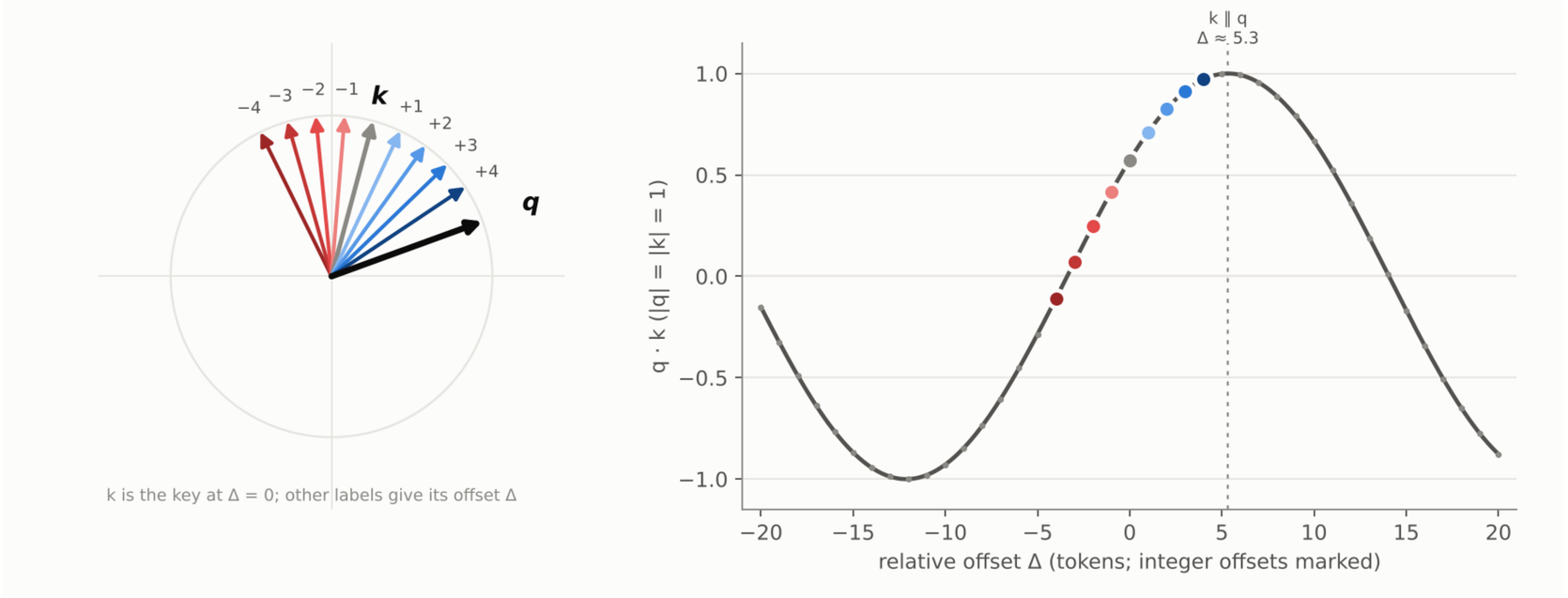
- Only the difference $\Delta = j - i$ matters:
  - rotate clockwise with $R_\theta^{-1} = R_{-\theta} = R_\theta^\top$
  - so $(R_{j\theta}k) \cdot (R_{i\theta}q) = R_{(j-i)\theta}k \cdot q$

$$R_\theta = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$

rotates CCW by θ

RoPE: one frequency

how do we get sin or cos out of dot product of embeddings?



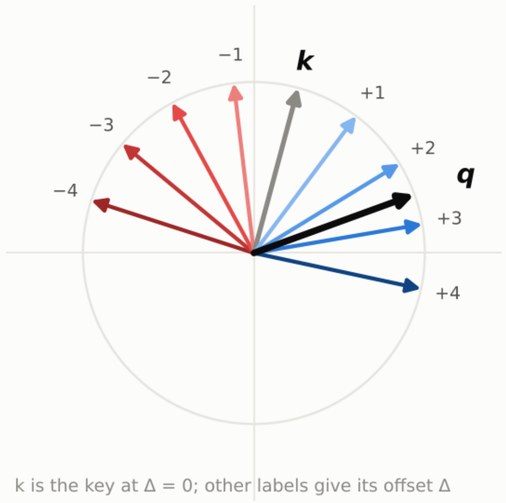
- Gives us a sinusoid:
  - frequency $= \theta$
  - amplitude (weight) $= \|q\| \, \|k\|$
  - phase = initial angle ϕ between q and k
- Called ***Rotary Positional Encoding***

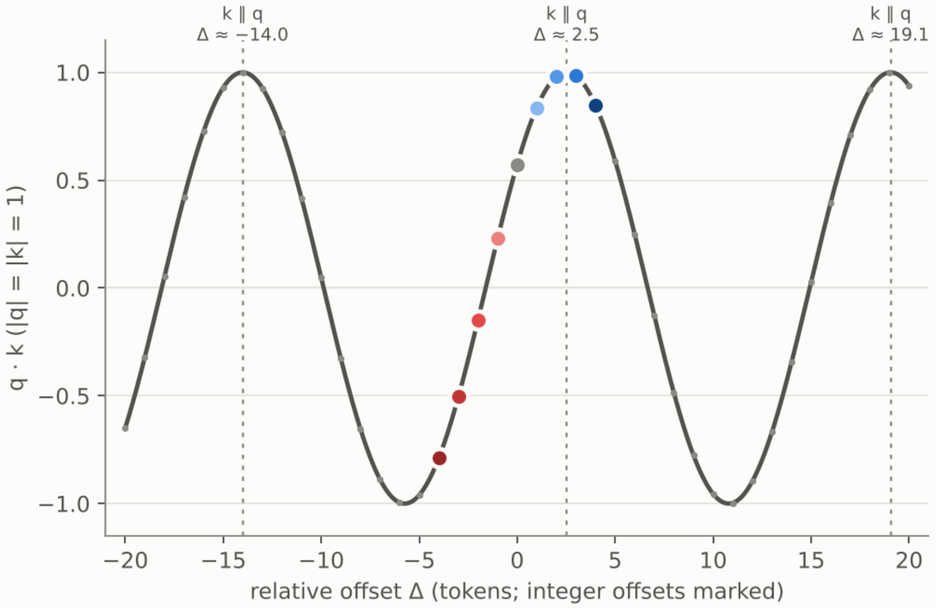
$$R_\theta = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$

rotates CCW by θ

RoPE: one frequency

how do we get sin or cos out of dot product of embeddings?





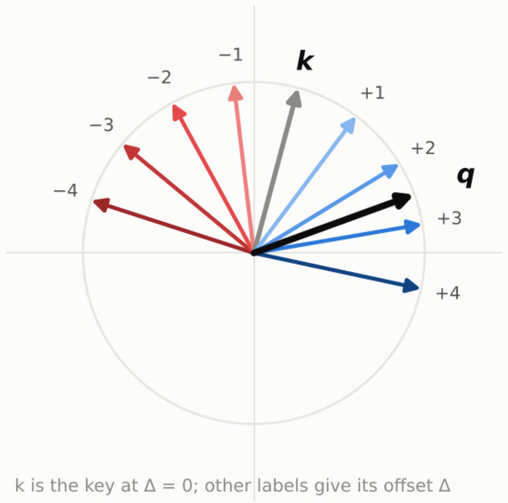
- Gives us a sinusoid:
  - frequency $= \theta$
  - amplitude (weight) $= \|q\| \, \|k\|$
  - phase = initial angle ϕ between q and k
- Called ***Rotary Positional Encoding***

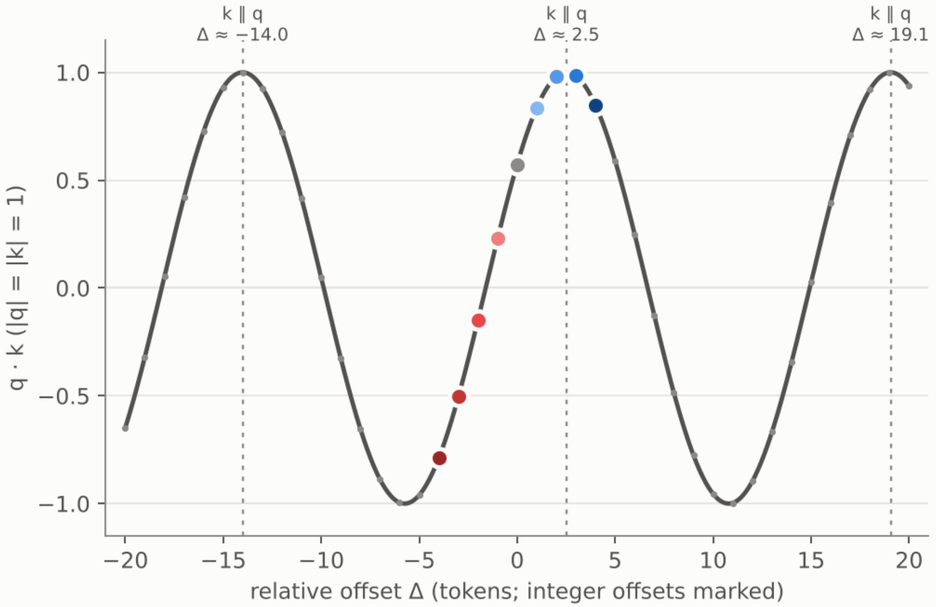
$$R_\theta = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$

rotates CCW by θ

RoPE: one frequency

how do we get sin or cos out of dot product of embeddings?





- Note it's equivalent to have both sin and cos with weights α, β or to have just sin with weight γ, phase ϕ:

$$\gamma \sin(\theta\Delta + \phi) = \gamma \sin(\theta\Delta)\cos(\phi) + \gamma \cos(\theta\Delta)\sin(\phi)$$
$$= \alpha \sin(\theta\Delta) + \beta \cos(\theta\Delta)$$

  - by angle-sum formula
  - polar ↔ Cartesian

$$R_\theta = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$

rotates CCW by θ

RoPE: multiple frequencies

$$q = \begin{pmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \\ q_5 \\ q_6 \\ \vdots \end{pmatrix} = \begin{pmatrix} q_{1:2} \\ \\ q_{3:4} \\ \\ q_{5:6} \\ \\ \vdots \end{pmatrix} \rightarrow q^{\text{enc}} = \begin{pmatrix} R_{i\theta_1} q_{1:2} \\ \\ R_{i\theta_2} q_{3:4} \\ \\ R_{i\theta_3} q_{5:6} \\ \\ \vdots \end{pmatrix} \qquad k = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \\ k_5 \\ k_6 \\ \vdots \end{pmatrix} = \begin{pmatrix} k_{1:2} \\ \\ k_{3:4} \\ \\ k_{5:6} \\ \\ \vdots \end{pmatrix} \rightarrow k^{\text{enc}} = \begin{pmatrix} R_{j\theta_1} k_{1:2} \\ \\ R_{j\theta_2} k_{3:4} \\ \\ R_{j\theta_3} k_{5:6} \\ \\ \vdots \end{pmatrix}$$

- Sample a geometric grid of frequencies $\theta_b = B^{\frac{b}{N}}$
- Divide q and k into blocks of 2 adjacent coordinates
- Encode by rotating block b by a multiple of θ_b
  - by same argument as above: attention score = sum of N sinusoids of different frequency, amplitude, and phase

RoPE: expressive power

$$q = \begin{pmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \\ q_5 \\ q_6 \\ \vdots \end{pmatrix} = \begin{pmatrix} q_{1:2} \\ \\ q_{3:4} \\ \\ q_{5:6} \\ \\ \vdots \end{pmatrix} \rightarrow q^{\text{enc}} = \begin{pmatrix} R_{i\theta_1} q_{1:2} \\ \\ R_{i\theta_2} q_{3:4} \\ \\ R_{i\theta_3} q_{5:6} \\ \\ \vdots \end{pmatrix} \qquad k = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \\ k_5 \\ k_6 \\ \vdots \end{pmatrix} = \begin{pmatrix} k_{1:2} \\ \\ k_{3:4} \\ \\ k_{5:6} \\ \\ \vdots \end{pmatrix} \rightarrow k^{\text{enc}} = \begin{pmatrix} R_{j\theta_1} k_{1:2} \\ \\ R_{j\theta_2} k_{3:4} \\ \\ R_{j\theta_3} k_{5:6} \\ \\ \vdots \end{pmatrix}$$

- N sinusoids of different frequency, amplitude, and phase
  - frequencies are fixed ahead of time: θ_b
  - for each q, can design a k that chooses amplitude and phase of each one $\rightarrow$ implements any attention score function $f(\Delta)$ to the limit of our basis (sin and cos at θ_b)
- Different q, k pairs have to share (increase error and/or use similar attention score functions)
  - also share with other functions of embedding

RoPE: expressive power

variant: partial RoPE — only rotate some blocks of q and k; remaining blocks keep globally useful information

$$q = \begin{pmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \\ q_5 \\ q_6 \\ \vdots \end{pmatrix} = \begin{pmatrix} q_{1:2} \\ \\ q_{3:4} \\ \\ q_{5:6} \\ \\ \vdots \end{pmatrix} \rightarrow q^{\text{enc}} = \begin{pmatrix} R_{i\theta_1} q_{1:2} \\ \\ R_{i\theta_2} q_{3:4} \\ \\ R_{i\theta_3} q_{5:6} \\ \\ \vdots \end{pmatrix} \qquad k = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \\ k_5 \\ k_6 \\ \vdots \end{pmatrix} = \begin{pmatrix} k_{1:2} \\ \\ k_{3:4} \\ \\ k_{5:6} \\ \\ \vdots \end{pmatrix} \rightarrow k^{\text{enc}} = \begin{pmatrix} R_{j\theta_1} k_{1:2} \\ \\ R_{j\theta_2} k_{3:4} \\ \\ R_{j\theta_3} k_{5:6} \\ \\ \vdots \end{pmatrix}$$

- N sinusoids of different frequency, amplitude, and phase
  - frequencies are fixed ahead of time: θ_b
  - for each q, can design a k that chooses amplitude and phase of each one → implements any attention score function $f(\Delta)$ to the limit of our basis (sin and cos at θ_b)
- Different q, k pairs have to share (increase error and/or use similar attention score functions)
  - also share with other functions of embedding

RoPE: expressive power

variant: partial RoPE — only rotate some blocks of q and k; remaining blocks keep globally useful information

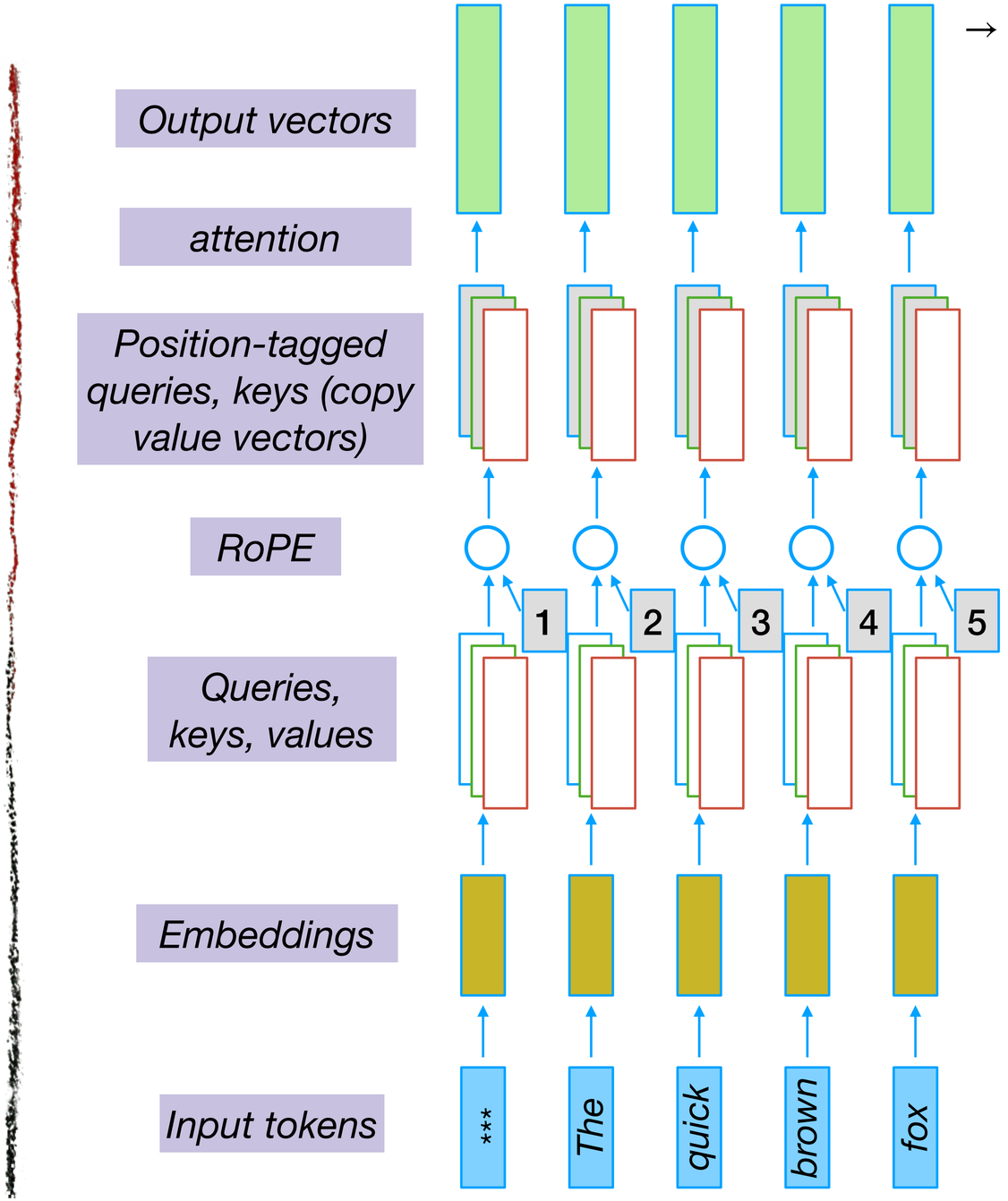
If fraction goes to zero, it's called NoPE

$$q = \begin{pmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \\ q_5 \\ q_6 \\ \vdots \end{pmatrix} = \begin{pmatrix} q_{1:2} \\ \\ q_{3:4} \\ \\ q_{5:6} \\ \\ \vdots \end{pmatrix} \rightarrow q^{\text{enc}} = \begin{pmatrix} R_{i\theta_1} q_{1:2} \\ \\ R_{i\theta_2} q_{3:4} \\ \\ R_{i\theta_3} q_{5:6} \\ \\ \vdots \end{pmatrix} \qquad k = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \\ k_5 \\ k_6 \\ \vdots \end{pmatrix} = \begin{pmatrix} k_{1:2} \\ \\ k_{3:4} \\ \\ k_{5:6} \\ \\ \vdots \end{pmatrix} \rightarrow k^{\text{enc}} = \begin{pmatrix} R_{j\theta_1} k_{1:2} \\ \\ R_{j\theta_2} k_{3:4} \\ \\ R_{j\theta_3} k_{5:6} \\ \\ \vdots \end{pmatrix}$$

- N sinusoids of different frequency, amplitude, and phase
  - frequencies are fixed ahead of time: θ_b
  - for each q, can design a k that chooses amplitude and phase of each one → implements any attention score function $f(\Delta)$ to the limit of our basis (sin and cos at θ_b)
- Different q, k pairs have to share (increase error and/or use similar attention score functions)
  - also share with other functions of embedding

Geoff Gordon

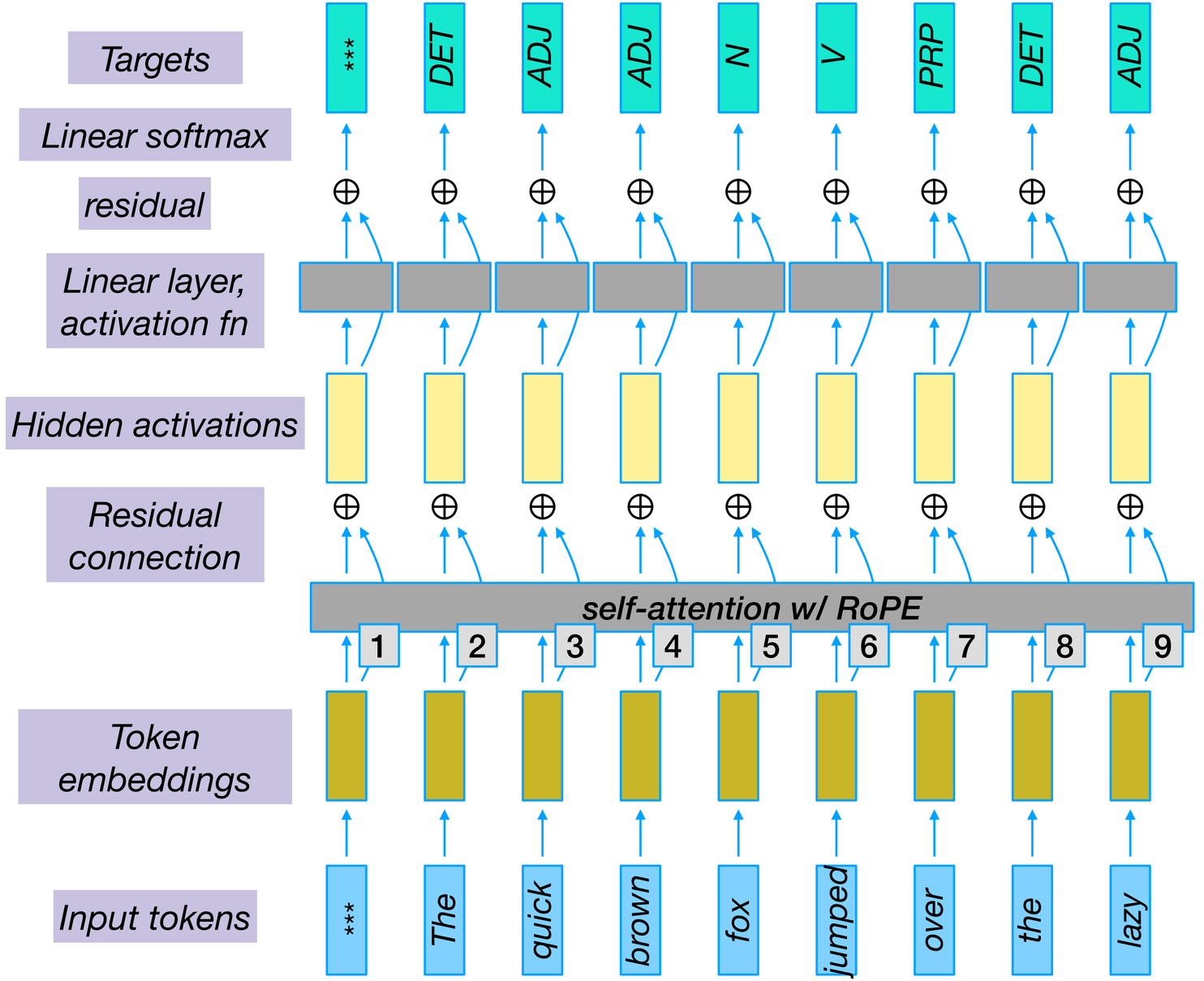
Attention with RoPE



- Put it together: look up embedding for each input token; compute queries, keys, and values w/ weight matrices K, Q, V; rotate blocks in each query/key to encode position (token index); dot product, softmax, and sum for attention

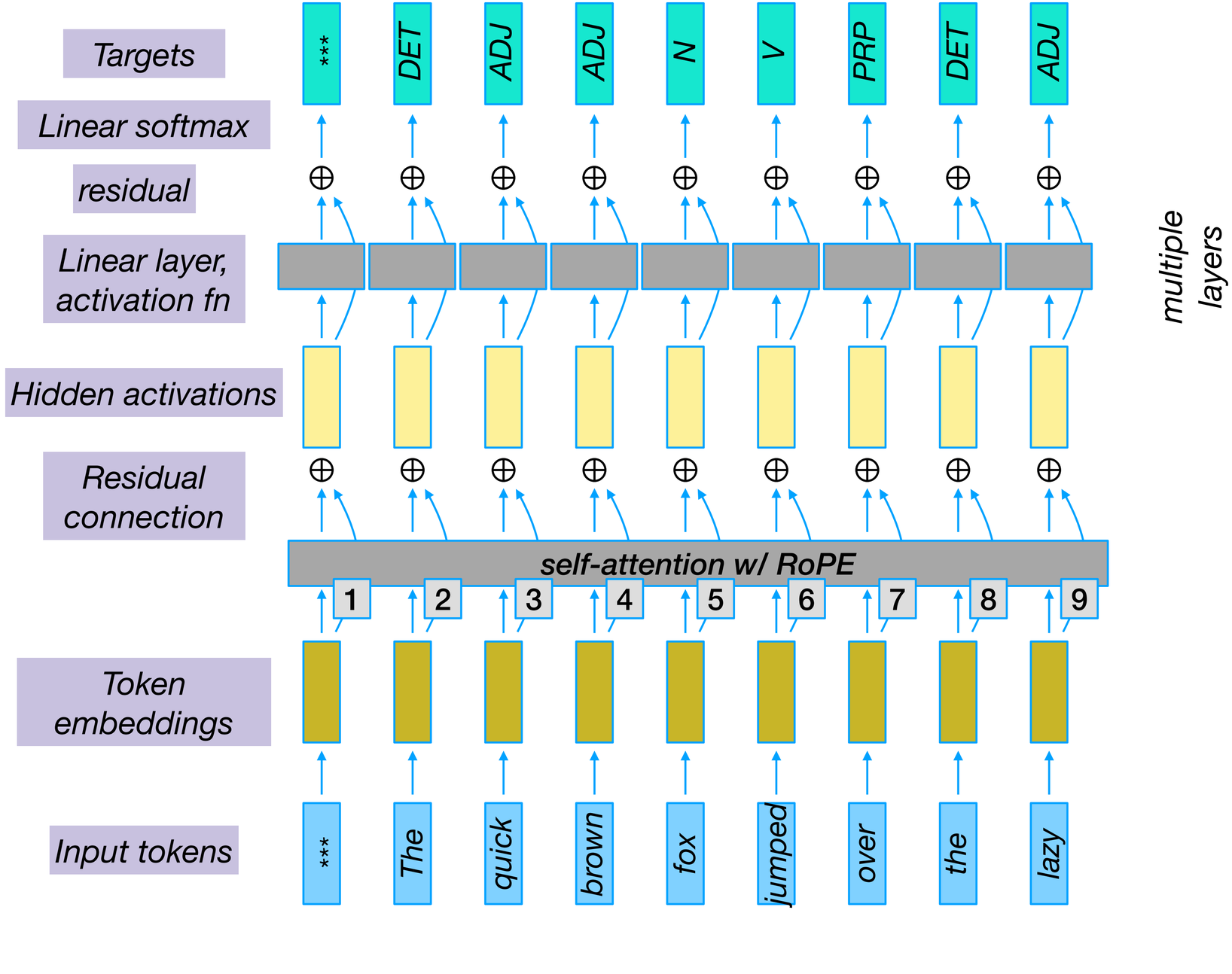
Predicting targets: pointwise MLP

Here, for POS tagging



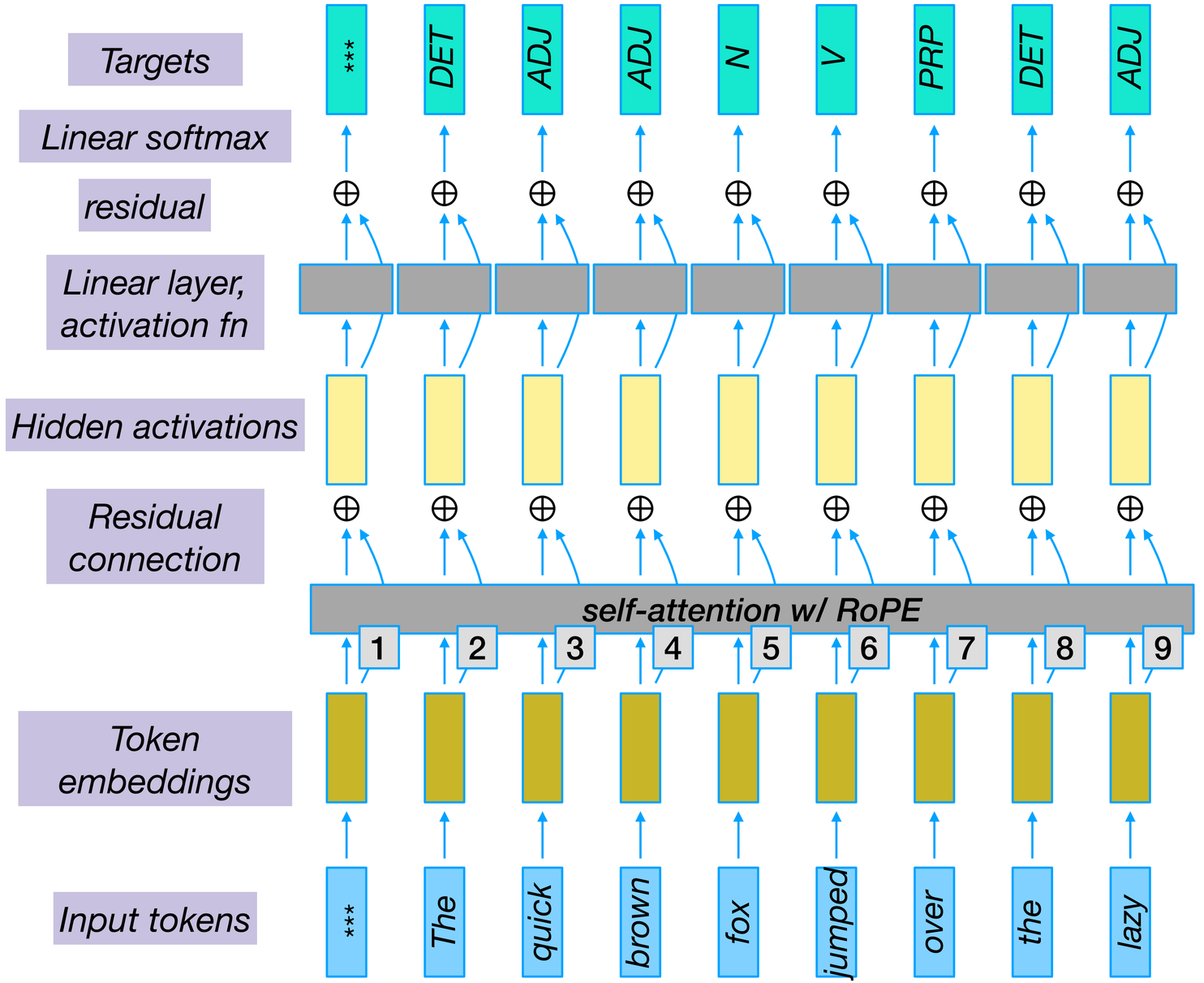
Predicting targets: pointwise MLP

Here, for POS tagging



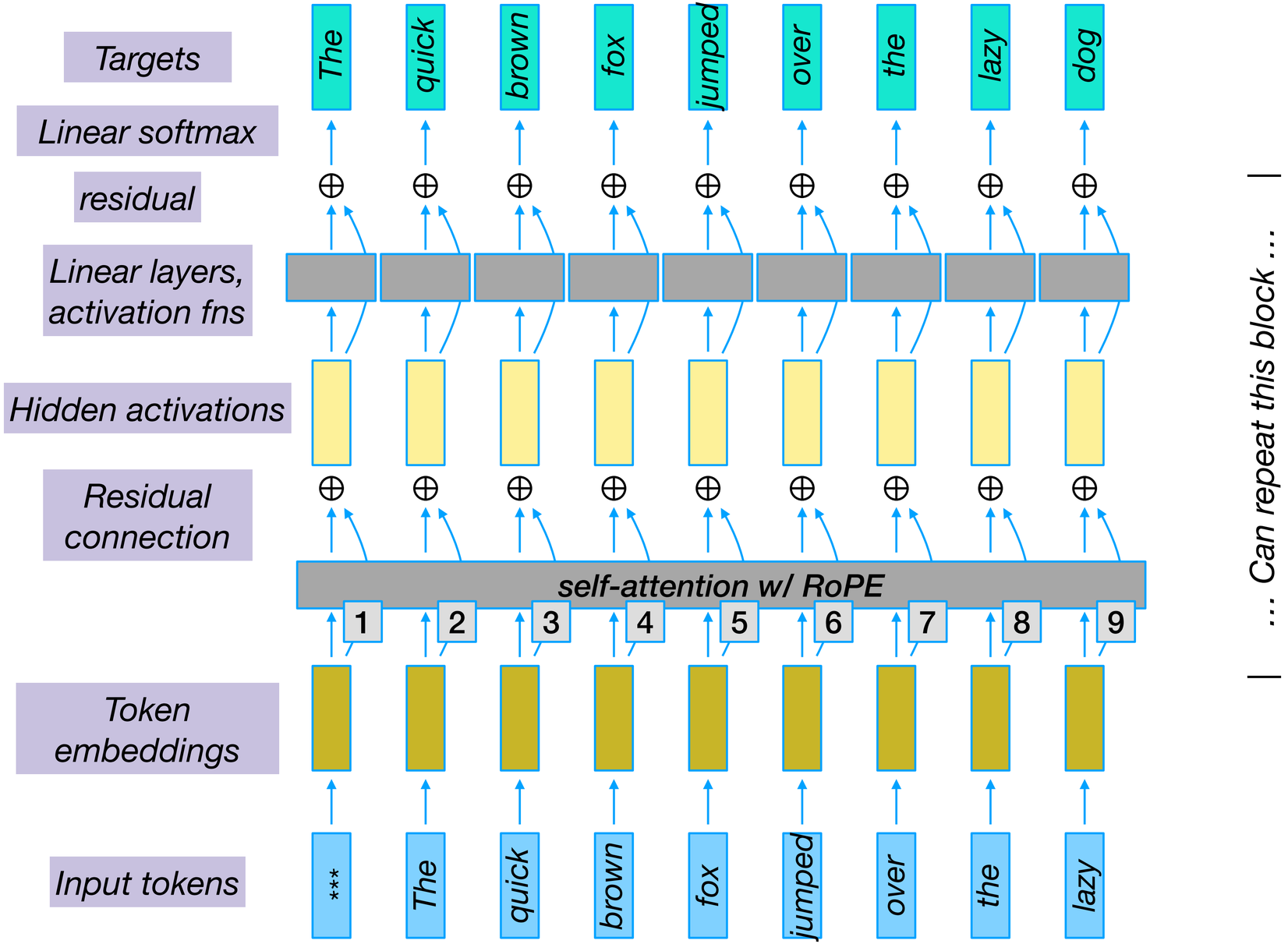
Predicting targets: pointwise MLP

Here, for POS tagging



What about next-token prediction?

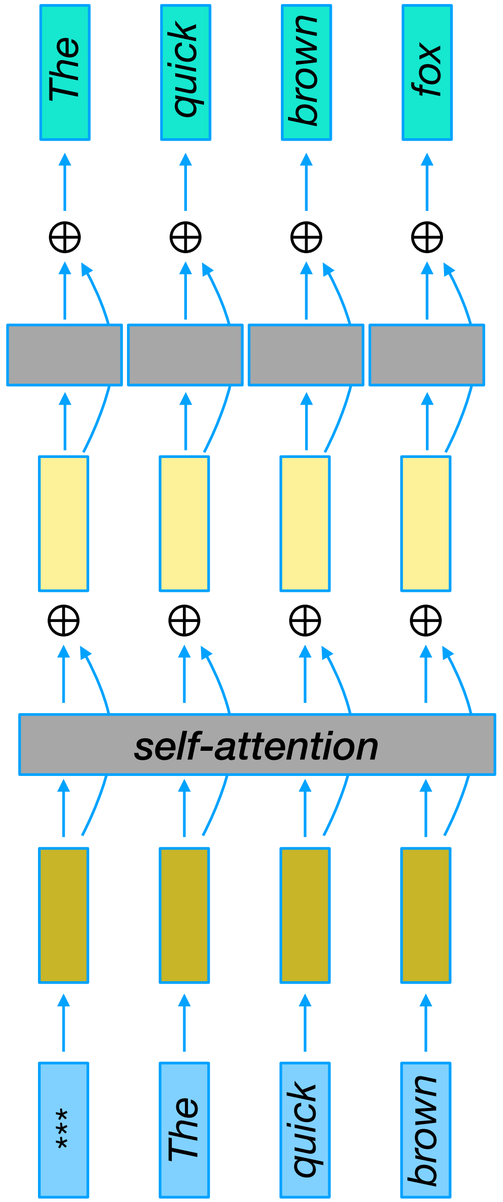
Can you spot the problem?



Attention mask

- Only let attention look at a subset of positions (*masked* attention)
- Here, own position and earlier in the sentence: causally masked self-attention

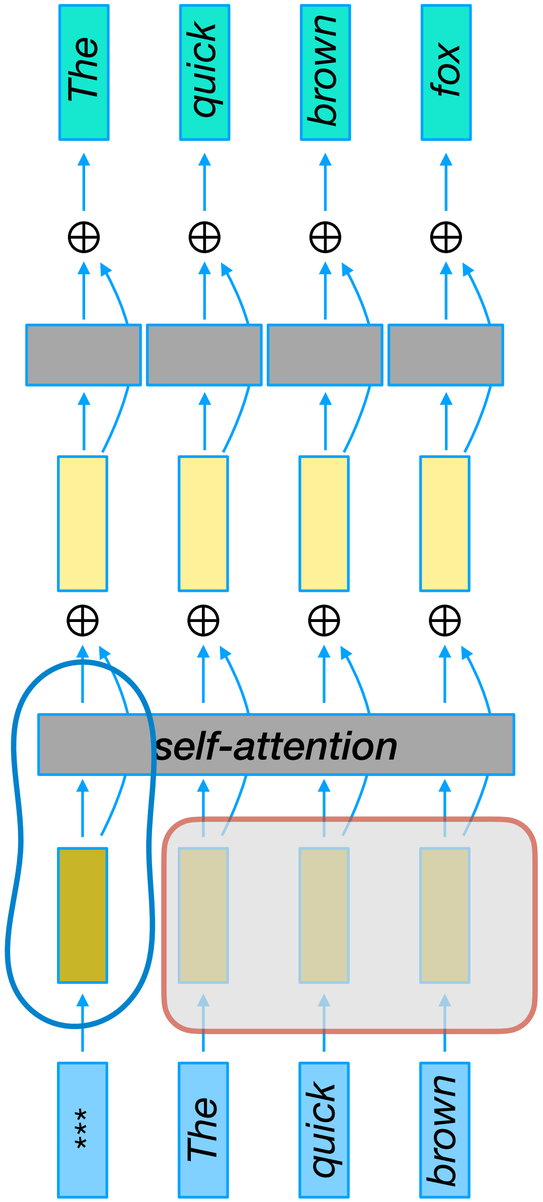
$$p_{ij} = \begin{cases} \dfrac{\exp(\frac{1}{\sqrt{d}} Qx_i \cdot Kx_j)}{\sum_{j'=1}^{i} \exp(\frac{1}{\sqrt{d}} Qx_i \cdot Kx_{j'})} & 1 \le j \le i \le T \\ 0 & \text{o/w} \end{cases}$$



Attention mask

- Only let attention look at a subset of positions (*masked* attention)
- Here, own position and earlier in the sentence: causally masked self-attention

$$p_{ij} = \begin{cases} \dfrac{\exp(\frac{1}{\sqrt{d}} Qx_i \cdot Kx_j)}{\sum_{j'=1}^{i} \exp(\frac{1}{\sqrt{d}} Qx_i \cdot Kx_{j'})} & 1 \le j \le i \le T \\ 0 & \text{o/w} \end{cases}$$

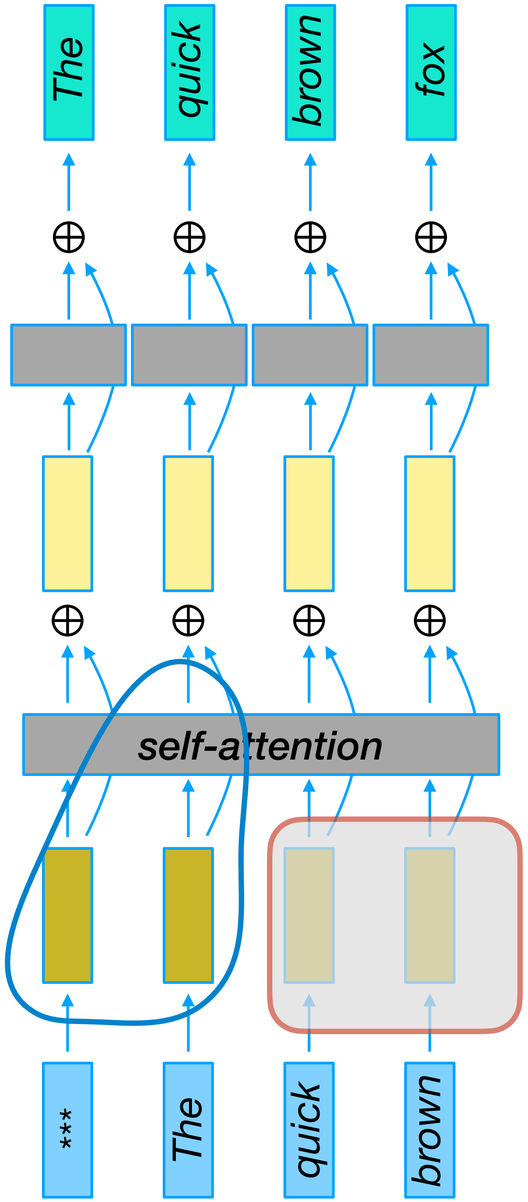


Geoff Gordon

Attention mask

- Only let attention look at a subset of positions (*masked* attention)
- Here, own position and earlier in the sentence: causally masked self-attention

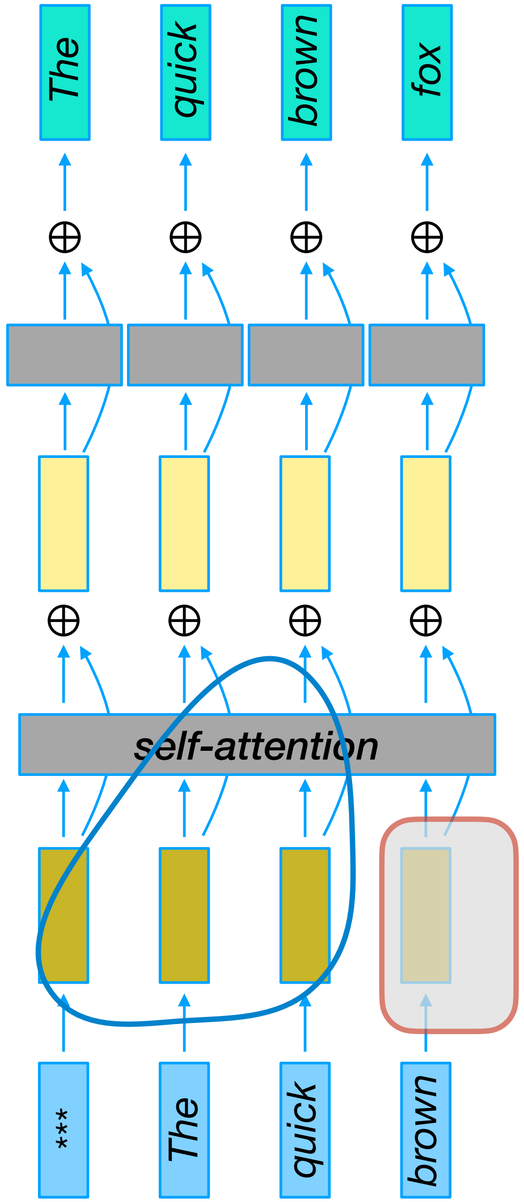
$$p_{ij} = \begin{cases} \dfrac{\exp(\frac{1}{\sqrt{d}} Q x_i \cdot K x_j)}{\sum_{j'=1}^{i} \exp(\frac{1}{\sqrt{d}} Q x_i \cdot K x_{j'})} & 1 \le j \le i \le T \\ 0 & \text{o/w} \end{cases}$$



Attention mask

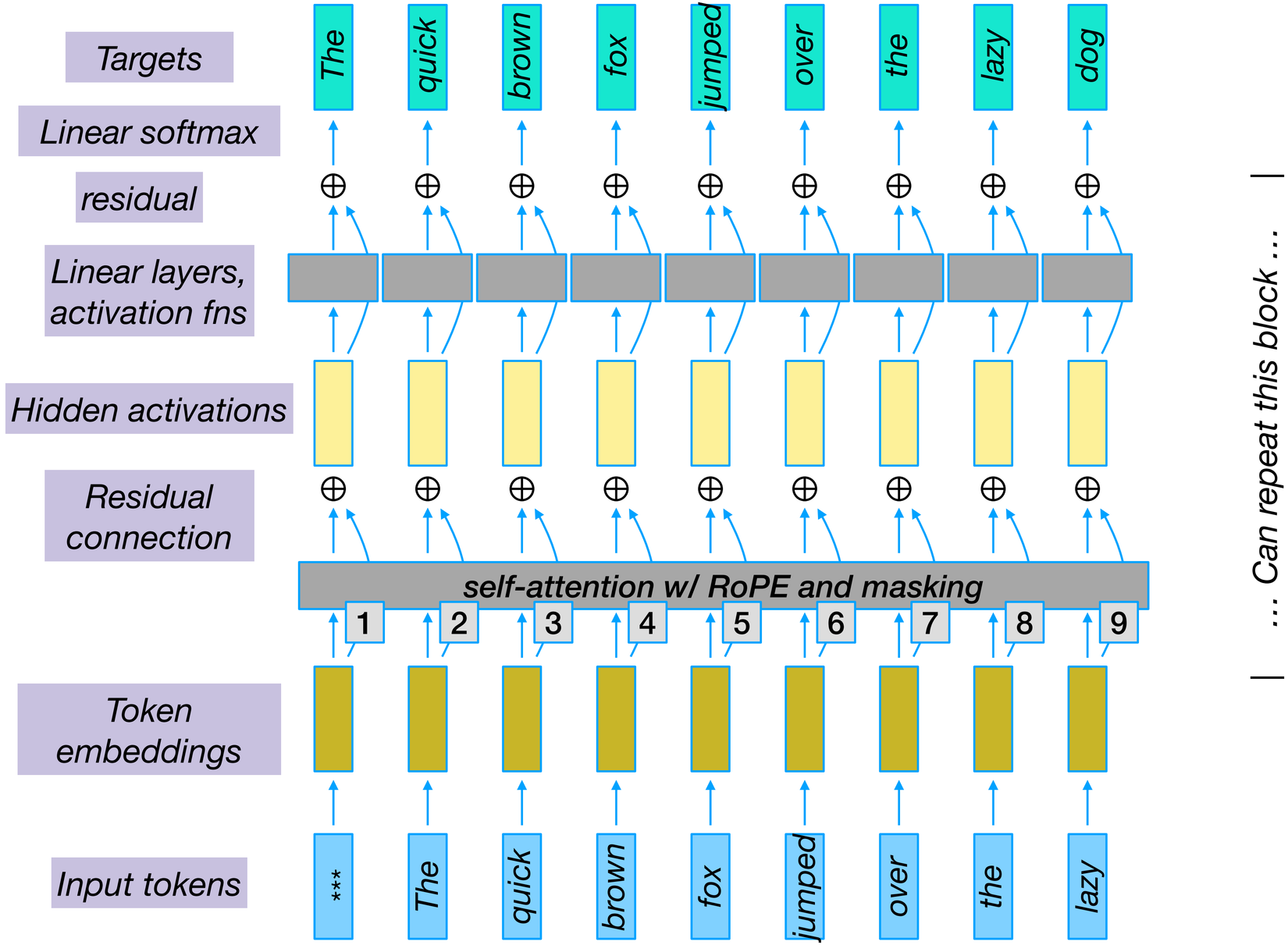
- Only let attention look at a subset of positions (*masked* attention)
- Here, own position and earlier in the sentence: causally masked self-attention

$$p_{ij} = \begin{cases} \dfrac{\exp(\frac{1}{\sqrt{d}} Q x_i \cdot K x_j)}{\sum_{j'=1}^{i} \exp(\frac{1}{\sqrt{d}} Q x_i \cdot K x_{j'})} & 1 \le j \le i \le T \\ 0 & \text{o/w} \end{cases}$$



Masked self-attention layers are perfect for next token prediction

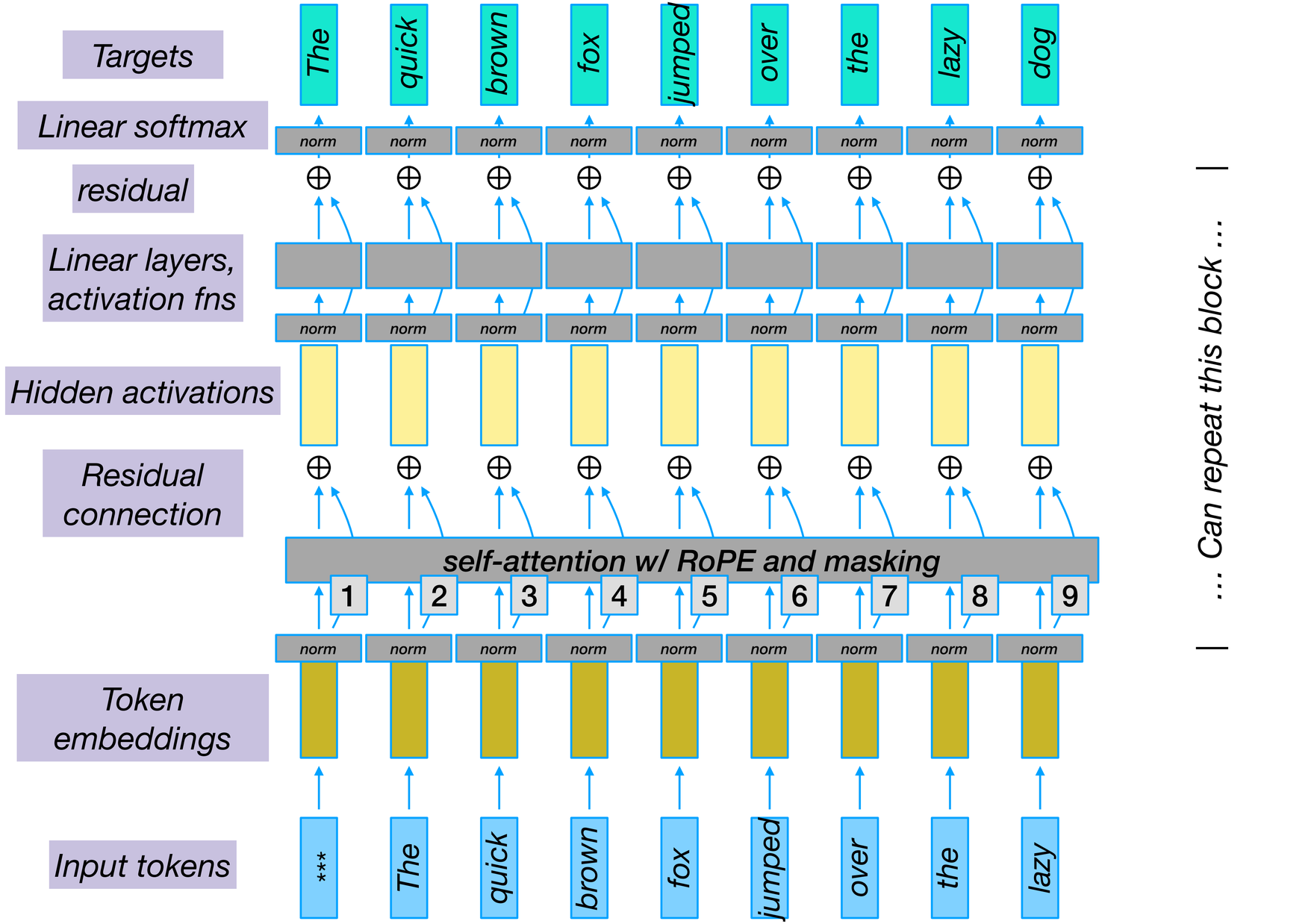
…with one more small modification



Transformer

This is LLaMA 3 if we use RMS norm, GQA, 2-layer MLP, SwiGLU

Original transformer used post-norm instead of pre-norm; positional encoding at embeddings



Transformer example

- Same tiny next-token problem as earlier: AABBA
- More complicated model than RNN, so the example leaves stuff out for simplicity
  - one attention head, 2d, no layer norm, no pointwise MLP
- Instead of sweeping along sequence, we process entire sequence in parallel and sweep through depth

A tiny transformer on {A, B}*

one layer, one attention head

embed	$e_t = E\,\mathrm{onehot}(x_t)$
query, key	$\bar{q}_t = W_Q e_t, \quad \bar{k}_t = W_K e_t$
RoPE	$q_t = R(t\theta)\,\bar{q}_t, \quad k_t = R(t\theta)\,\bar{k}_t$
value	$v_t = W_V e_t$
attend	$\alpha_{tj} = \mathrm{softmax}_{j \le t}(q_t \cdot k_j)$
mix	$z_t = \sum_{j \le t} \alpha_{tj} v_j$
residual	$h_t = e_t + z_t$
read out	$o_t = U h_t + c$
predict	$p_t = \mathrm{softmax}(o_t)$
loss	$L = \sum_t -y_t \cdot \ln p_t, \quad y_t = \mathrm{onehot}(x_{t+1})$

$R(t\theta)$ rotates by $t\theta$: position t turns its query and key by $t \times 45°$.
The score $q_t \cdot k_j$ then depends on the positions only through the gap $t - j$.

Left out to fit on a slide: LayerNorm, the MLP block, multiple heads and layers.
W_O is folded into W_V, and the usual $1/\sqrt{d}$ score scaling into W_Q.

Parameters

E

$$\begin{bmatrix} 1.00 & 0.00 \\ 0.00 & 1.00 \end{bmatrix}$$

e_A e_B

columns are the embeddings;
here simply the two axes

W_Q

$$\begin{bmatrix} 1.00 & 4.00 \\ -2.00 & -2.00 \end{bmatrix}$$

W_K

$$\begin{bmatrix} 1.00 & 1.00 \\ 0.00 & 0.00 \end{bmatrix}$$

W_V

$$\begin{bmatrix} 0.00 & 0.00 \\ 1.00 & -1.00 \end{bmatrix}$$

U

logit A
logit B

$$\begin{bmatrix} 0.00 & -1.00 \\ 0.00 & 1.00 \end{bmatrix}$$

c

$$\begin{bmatrix} 1.00 \\ 0.00 \end{bmatrix}$$

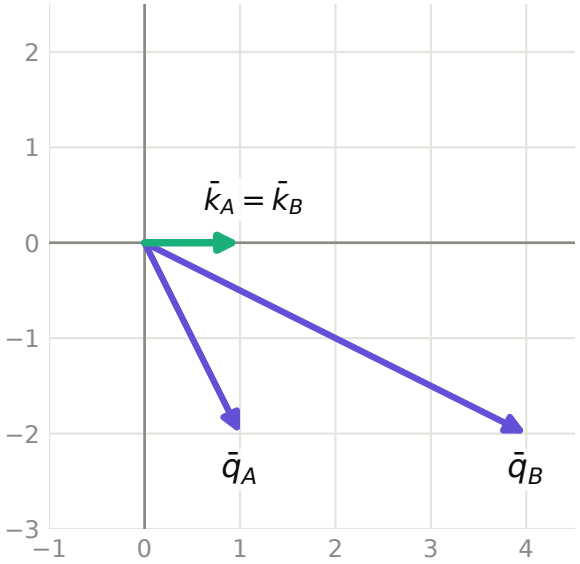
$\theta = 45°$

No positional vectors are added: position enters only through
the rotation of queries and keys.

What the parameters do

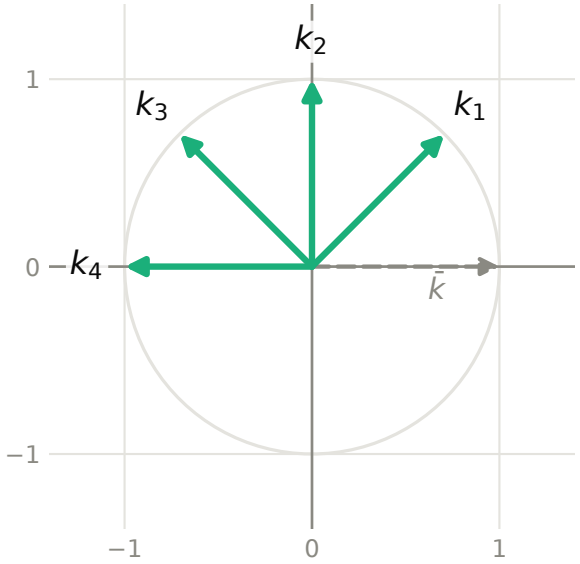
each panel shows one part of the head

Before rotation: W_Q, W_K



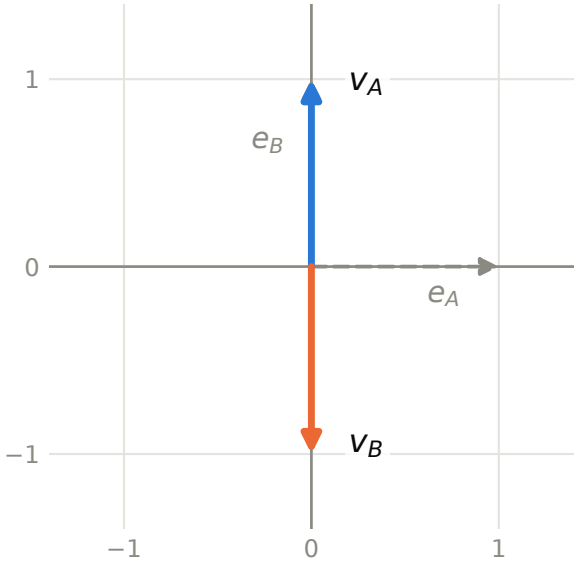
W_K sends both tokens to the same key, (1, 0): keys carry no token information. Both queries point down and to the right, about one rotation step (45°) behind the key direction.

RoPE: the key at position t



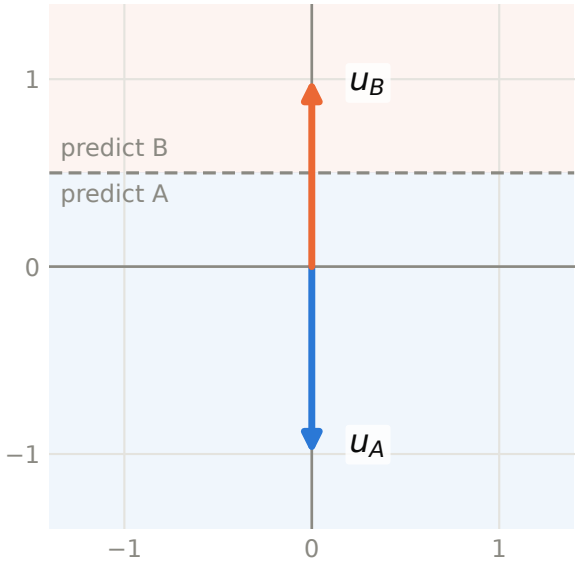
Position t turns its key by $t \times 45°$ (and its query by the same angle). A query that sits one step behind its own position lines up best with the previous key: a previous-token head.

Values: W_V



What each token contributes when attended to (dashed: the embeddings, which are just the axes). v_A pushes unit 2 up, v_B pushes it down. Values depend only on the token.

Read-out: rows of U



Each logit is h_t dotted with a row of U, plus c. Dashed: where the two logits tie. Above it the model predicts B, below it A.

Forward 1: queries, keys and values

every position at once

inputs A A B B A — all four positions are processed at once; each predicts the next token (the last token is only a target)

1 2 3 4

queries

$\bar{q}_t = W_Q e_t, \quad q_t = R(t\theta)\,\bar{q}_t$

		e_t	W_Q	$\bar{q}_t$	rotate		q_t
1	A	1.00 0.00	→	1.00 -2.00	→	1 A 1θ = 45°	2.12 -0.71
2	A	1.00 0.00		1.00 -2.00		2 A 2θ = 90°	2.00 1.00
3	B	0.00 1.00		4.00 -2.00		3 B 3θ = 135°	-1.41 4.24
4	B	0.00 1.00		4.00 -2.00		4 B 4θ = 180°	-4.00 2.00

keys

$\bar{k}_t = W_K e_t, \quad k_t = R(t\theta)\,\bar{k}_t$

		e_t	W_K	$\bar{k}_t$	rotate		k_t
1	A	1.00 0.00	→	1.00 0.00	→	1 A 1θ = 45°	0.71 0.71
2	A	1.00 0.00		1.00 0.00		2 A 2θ = 90°	0.00 1.00
3	B	0.00 1.00		1.00 0.00		3 B 3θ = 135°	-0.71 0.71
4	B	0.00 1.00		1.00 0.00		4 B 4θ = 180°	-1.00 0.00

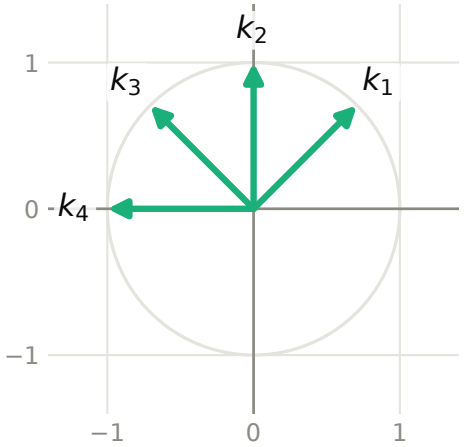
values

$v_t = W_V e_t$ (no rotation)

		e_t	W_V	v_t
1	A	1.00 0.00	→	0.00 1.00
2	A	1.00 0.00		0.00 1.00
3	B	0.00 1.00		0.00 -1.00
4	B	0.00 1.00		0.00 -1.00

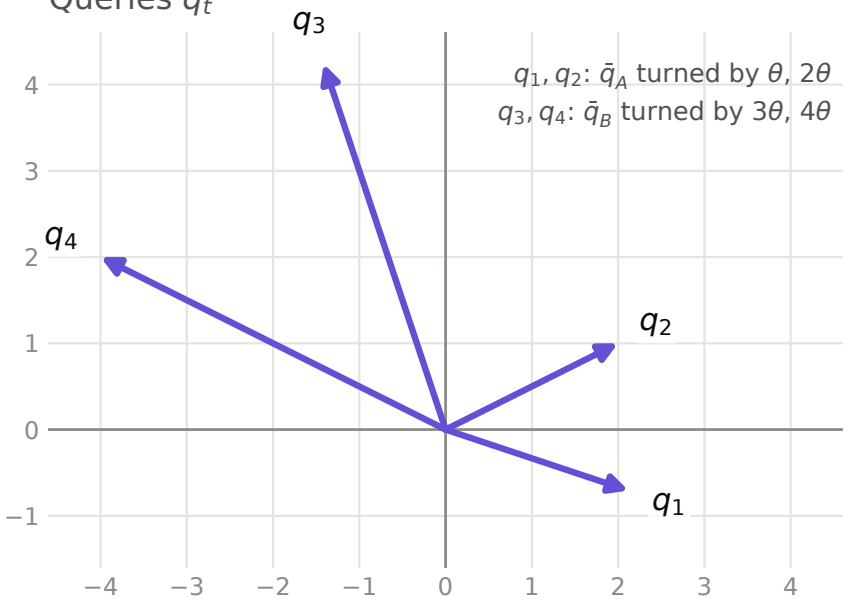
Values skip the rotation, so they depend only on the token: $v_1 = v_2 = v_A$ and $v_3 = v_4 = v_B$.

Keys k_t



All four keys are the same vector (1, 0), turned 45° per position.

Queries q_t



Forward 2: attention scores and weights

query position t looks at key positions $j \le t$

inputs **A** **A** **B** **B** **A**

1 2 3 4

all four positions are processed at once; each predicts the next token (the last token is only a target)

scores $q_t \cdot k_j$ (key position $j \rightarrow$)

query t		1	2	3	4
1	A	1.00			
2	A	2.12	1.00		
3	B	2.00	4.24	4.00	
4	B	-1.41	2.00	4.24	4.00

softmax each row →

weights α_{tj}

		1	2	3	4
1	A	1.00			
2	A	0.75	0.25		
3	B	0.06	0.53	0.41	
4	B	0.00	0.06	0.53	0.41

Grey cells are masked: a position cannot look ahead. Every row's largest weight sits one step left of the diagonal, on the previous token. The rest goes mostly to the position itself.

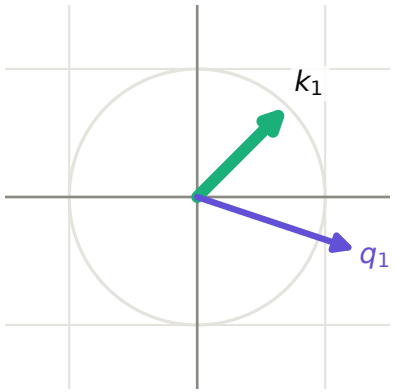
Row 4, worked out:

$\alpha_{4,\cdot} = \text{softmax}(-1.41,\ 2.00,\ 4.24,\ 4.00)$

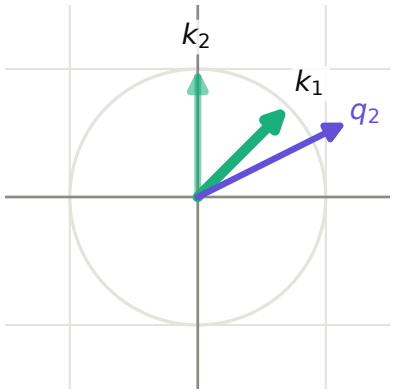
$= (0.00, 0.06, 0.53, 0.41)$

e.g. $q_4 \cdot k_3$ = (-4.00)(-0.71) + (2.00)(0.71) = 4.24

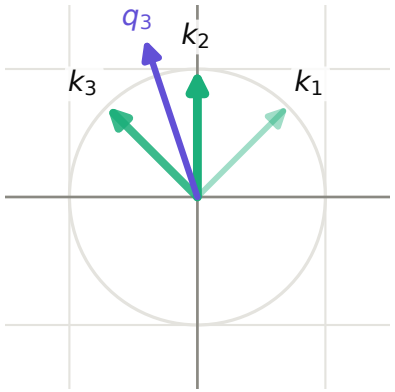
query 1 (A)



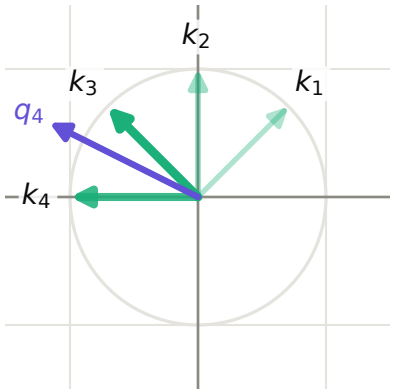
query 2 (A)



query 3 (B)



query 4 (B)



Queries drawn at a fixed length (only their direction matters for the ranking, since every key has length 1). Thicker key = more weight.

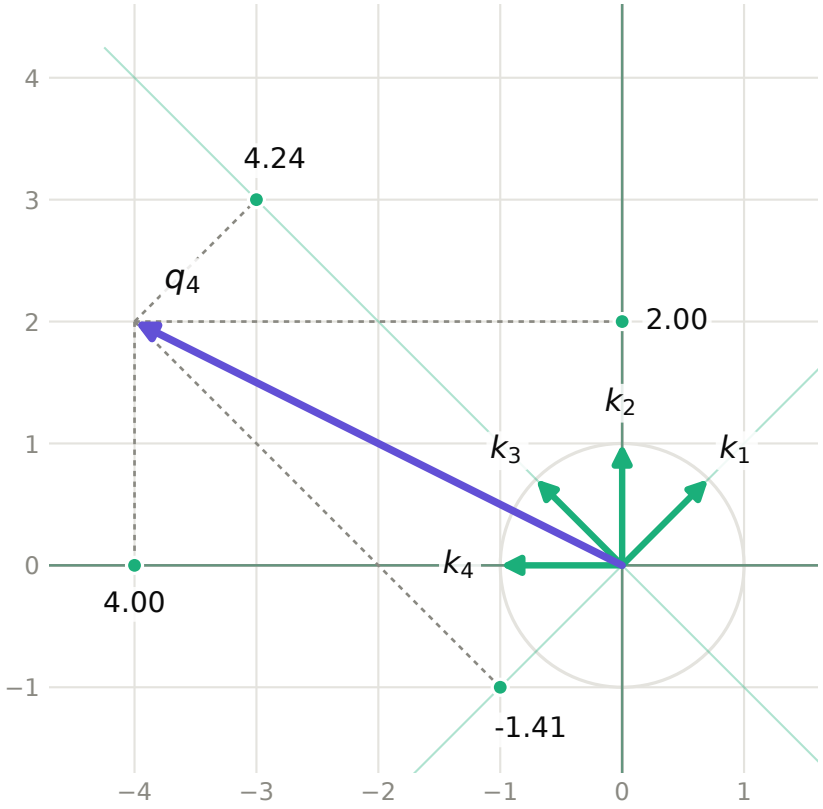
Forward 3: what position 4 reads

query q_4 against keys $k_1, \ldots, k_4$, then a mix of values

inputs A A B B A — position 4 reads B and must predict A

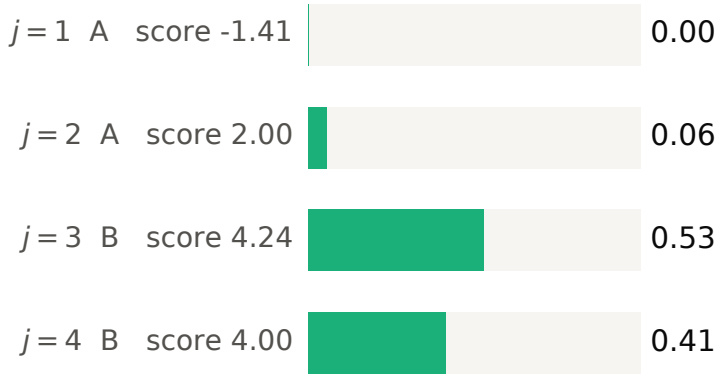
1 2 3 4

1. Scores: project q_4 onto each key

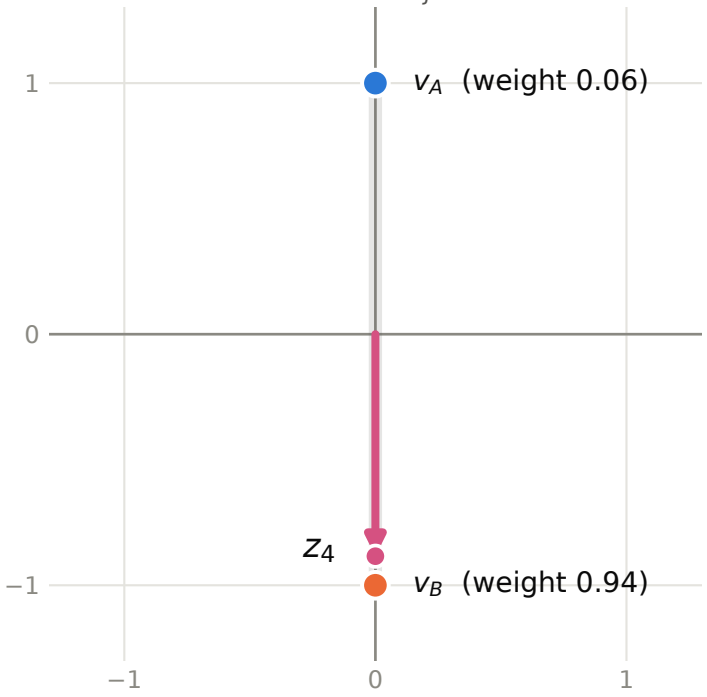


Each key has length 1, so $q_4 \cdot k_j$ is how far along k_j's line the dotted drop from q_4 lands (negative: behind the origin).

2. Weights α_{4j} = softmax of the scores



3. Mix the values: $z_4 = \sum_j \alpha_{4j} v_j$



Weight on A positions (1, 2): 0.06; on B (3, 4): 0.94.
z_4 = 0.06 v_A + 0.94 v_B = (0.00, -0.88).
Values depend only on the token, so every z_t lies on the grey segment.

Forward 4: residual, read-out and loss

total loss L = 2.24

inputs **A** **A** **B** **B** **A**

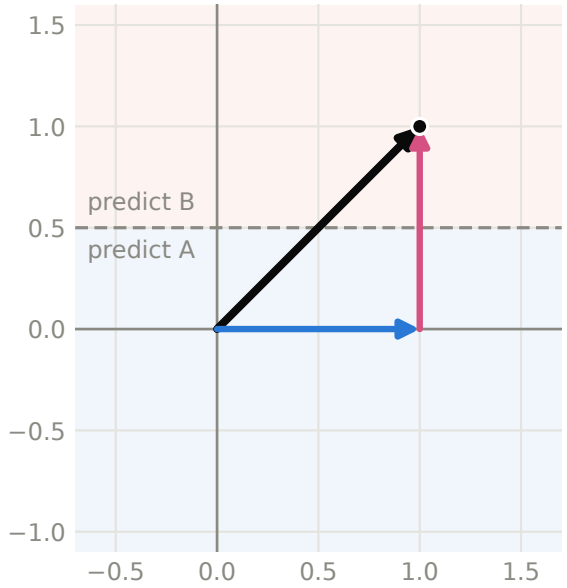
1 2 3 4

all four positions are processed at once; each predicts the next token (the last token is only a target)

	e_t		z_t		h_t		$o_t = Uh_t + c$		p_t (A, B)		$L_t = -\ln p_t[x_{t+1}]$
1 **A**	1.00	0.00	0.00	1.00	1.00	1.00	0.00	1.00	0.27	0.73	1.31
2 **A**	1.00	0.00	0.00	1.00	1.00	1.00	0.00	1.00	0.27	0.73	0.31
3 **B**	0.00	1.00	0.00	0.17	0.00	1.17	-0.17	1.17	0.21	0.79	0.23
4 **B**	0.00	1.00	0.00	-0.88	0.00	0.12	0.88	0.12	0.68	0.32	0.38

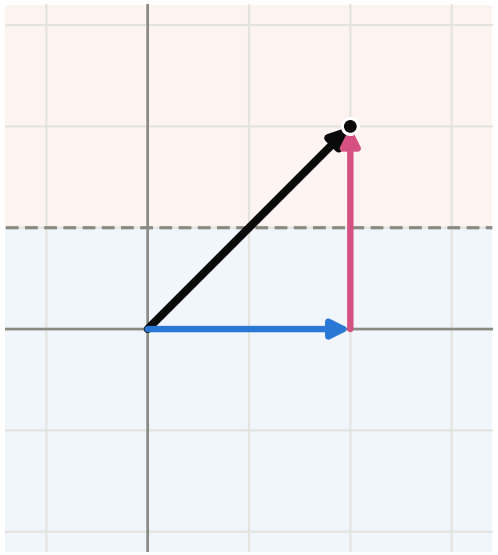
Each position predicts its own next token; the losses add up.

t = 1, reads A



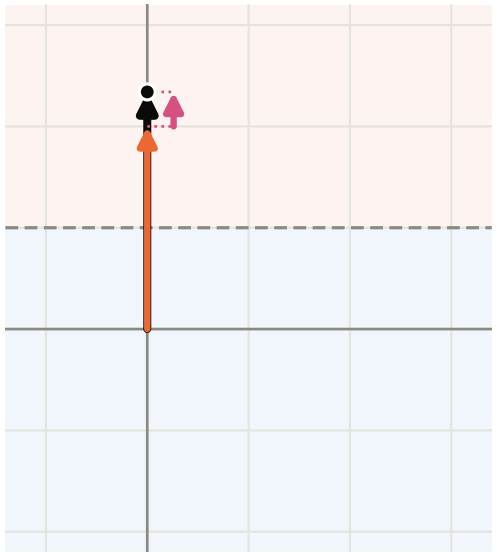
p(A) 0.27
p(B) 0.73
target A
L_1 = 1.31

t = 2, reads A



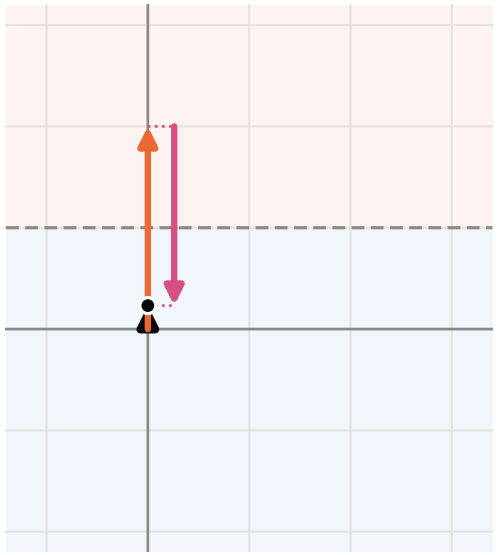
p(A) 0.27
p(B) 0.73
target B
L_2 = 0.31

t = 3, reads B



p(A) 0.21
p(B) 0.79
target B
L_3 = 0.23

t = 4, reads B



p(A) 0.68
p(B) 0.32
target A
L_4 = 0.38

e_t (token colour) $+ z_t$ from attention $= h_t$

Shading: the read-out's prediction. At t = 1 attention can only return v_A.

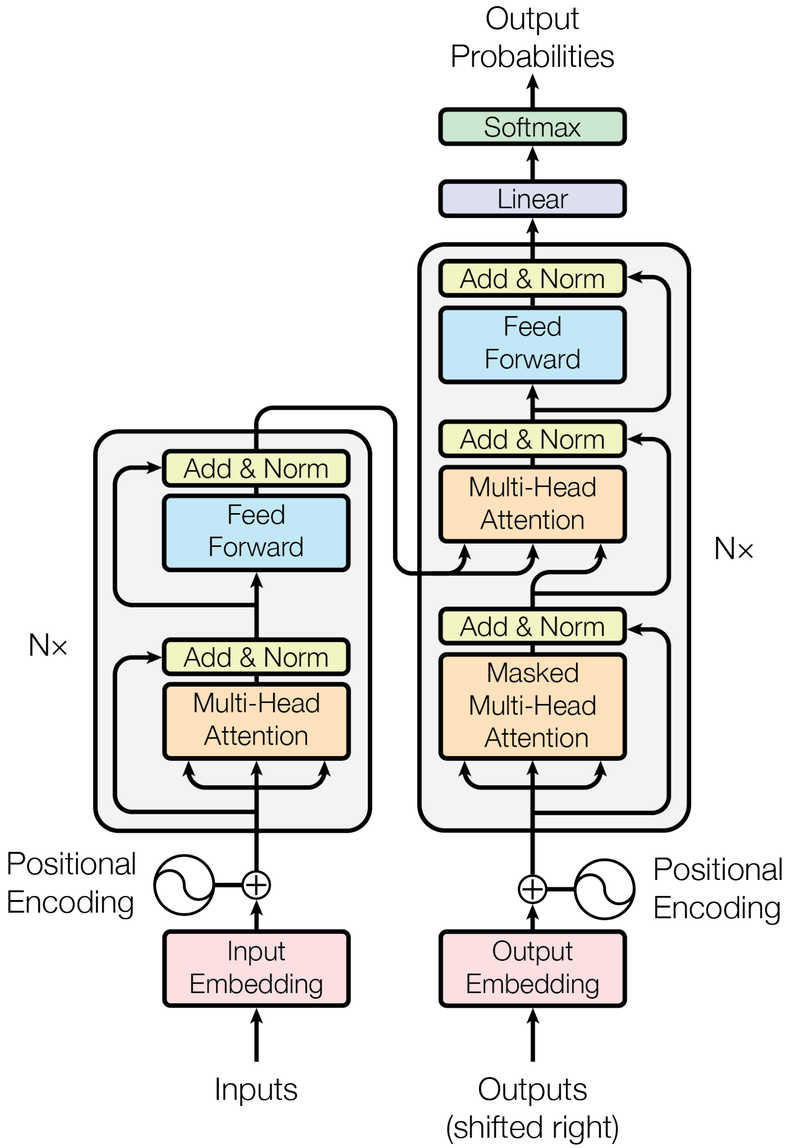
Transduction, generation, classification

- Above we designed a transformer for a *transduction* task (each input has a corresponding output)
  - then used it for *generation* by masking, shifting tokens after each step and appending outputs
- For a many:1 task like classification, we can add a distinguished token *** at beginning or end, read result off of corresponding output token
  - turn off masking
  - beginning and end are symmetric, doesn't matter which

Encoder-decoder

not as common any more, since we can just increase context length instead

- If we have side info that we always want available to attend to, can use a two-part network:
- Encoder looks at side info, outputs a DB of embeddings
  - can use unmasked (bidirectional) attention
- Decoder attends over encoder's DB plus its own (backprop through both)
  - unmasked attention to encoder
  - decoder can use masking and shifting for completion
- Previous architecture is often called *decoder-only* since it is analogous to decoder half



Why transformers?

- Flexible: variable-length inputs, multimodal data, enables large-scale search (e.g., RAG)
- Capable: handle a wide range of ML problems well
  - this wasn't a priori obvious: e.g., can construct reasonable problems where RNNs provably do better
- Long context: don't forget distant past even while being accurate and detailed in the short term (cf. RNNs)
  - lets us provide, produce, and benefit from lots of info at once: extra examples, rules/policies, complex reasoning
- Scaling laws
  - experiments on small models let us design successful larger models
  - e.g., tune hyperparameters, test model features

Why transformers?

- Massively parallel (both forward and backward pass)
  - each token position is completely independent outside of attention — need only infrequent sync w/ other positions
  - sparse and linear attention variants, even more so
- Matches well w/ current architectures (clusters of GPU machines): very heavy on tensor ops (einsums), less extreme on communication latency and bandwidth
- Parallelism lets us train huge models on huge datasets
  - store a lot of information in 100B weights
  - gives access to long tail (rare but useful info)
  - lets us average out noise ($1/\sqrt{n}$ rate)

What else?

- We saw the complete Llama 3 architecture (2024)
- More recent architectures add a couple more ideas
- E.g., Qwen 3.6 (March 2026):
  - ***mixture of experts*** (MoE): activate only part of model (~10% of weights) per token (***router*** learns what part)
  - allows model to store more information (more weights) without increasing computation
  - ***linear attention***: simplified version of key→value store using only matrix multiplication, time $O(T)$ for T tokens
  - tradeoff is lossy memory, particularly for distant past — can replace 75% of attention layers, remaining 25% "real" attention can still do fine-grained search of past